

Linear Algebra Review and Reference

Zico Kolter (updated by Chuong Do)

September 30, 2015

Contents

1	Basic Concepts and Notation	2
1.1	Basic Notation	2
2	Matrix Multiplication	3
2.1	Vector-Vector Products	4
2.2	Matrix-Vector Products	4
2.3	Matrix-Matrix Products	5
3	Operations and Properties	7
3.1	The Identity Matrix and Diagonal Matrices	8
3.2	The Transpose	8
3.3	Symmetric Matrices	8
3.4	The Trace	9
3.5	Norms	10
3.6	Linear Independence and Rank	11
3.7	The Inverse	11
3.8	Orthogonal Matrices	12
3.9	Range and Nullspace of a Matrix	12
3.10	The Determinant	14
3.11	Quadratic Forms and Positive Semidefinite Matrices	17
3.12	Eigenvalues and Eigenvectors	18
3.13	Eigenvalues and Eigenvectors of Symmetric Matrices	19
4	Matrix Calculus	20
4.1	The Gradient	20
4.2	The Hessian	22
4.3	Gradients and Hessians of Quadratic and Linear Functions	23
4.4	Least Squares	25
4.5	Gradients of the Determinant	25
4.6	Eigenvalues as Optimization	26

1 Basic Concepts and Notation

Linear algebra provides a way of compactly representing and operating on sets of linear equations. For example, consider the following system of equations:

$$\begin{array}{rcl} 4x_1 & - & 5x_2 = -13 \\ -2x_1 & + & 3x_2 = 9. \end{array}$$

This is two equations and two variables, so as you know from high school algebra, you can find a unique solution for x_1 and x_2 (unless the equations are somehow degenerate, for example if the second equation is simply a multiple of the first, but in the case above there is in fact a unique solution). In matrix notation, we can write the system more compactly as

$$Ax = b$$

with

$$A = \begin{bmatrix} 4 & -5 \\ -2 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} -13 \\ 9 \end{bmatrix}.$$

As we will see shortly, there are many advantages (including the obvious space savings) to analyzing linear equations in this form.

1.1 Basic Notation

We use the following notation:

- By $A \in \mathbb{R}^{m \times n}$ we denote a matrix with m rows and n columns, where the entries of A are real numbers.
- By $x \in \mathbb{R}^n$, we denote a vector with n entries. By convention, an n -dimensional vector is often thought of as a matrix with n rows and 1 column, known as a **column vector**. If we want to explicitly represent a **row vector** — a matrix with 1 row and n columns — we typically write x^T (here x^T denotes the transpose of x , which we will define shortly).
- The i th element of a vector x is denoted x_i :

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

- We use the notation a_{ij} (or A_{ij} , $A_{i,j}$, etc) to denote the entry of A in the i th row and j th column:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

- We denote the j th column of A by a_j or $A_{:,j}$:

$$A = \begin{bmatrix} | & | & \cdots & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & \cdots & | \end{bmatrix}.$$

- We denote the i th row of A by a_i^T or $A_{i,:}$:

$$A = \begin{bmatrix} - & a_1^T & - \\ - & a_2^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix}.$$

- Note that these definitions are ambiguous (for example, the a_1 and a_1^T in the previous two definitions are *not* the same vector). Usually the meaning of the notation should be obvious from its use.

2 Matrix Multiplication

The product of two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$ is the matrix

$$C = AB \in \mathbb{R}^{m \times p},$$

where

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

Note that in order for the matrix product to exist, the number of columns in A must equal the number of rows in B . There are many ways of looking at matrix multiplication, and we'll start by examining a few special cases.

2.1 Vector-Vector Products

Given two vectors $x, y \in \mathbb{R}^n$, the quantity $x^T y$, sometimes called the **inner product** or **dot product** of the vectors, is a real number given by

$$x^T y \in \mathbb{R} = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \sum_{i=1}^n x_i y_i.$$

Observe that inner products are really just special case of matrix multiplication. Note that it is always the case that $x^T y = y^T x$.

Given vectors $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$ (not necessarily of the same size), $xy^T \in \mathbb{R}^{m \times n}$ is called the **outer product** of the vectors. It is a matrix whose entries are given by $(xy^T)_{ij} = x_i y_j$, i.e.,

$$xy^T \in \mathbb{R}^{m \times n} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix} = \begin{bmatrix} x_1 y_1 & x_1 y_2 & \cdots & x_1 y_n \\ x_2 y_1 & x_2 y_2 & \cdots & x_2 y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_m y_1 & x_m y_2 & \cdots & x_m y_n \end{bmatrix}.$$

As an example of how the outer product can be useful, let $\mathbf{1} \in \mathbb{R}^n$ denote an n -dimensional vector whose entries are all equal to 1. Furthermore, consider the matrix $A \in \mathbb{R}^{m \times n}$ whose columns are all equal to some vector $x \in \mathbb{R}^m$. Using outer products, we can represent A compactly as,

$$A = \begin{bmatrix} | & | & \cdots & | \\ x & x & \cdots & x \\ | & | & \cdots & | \end{bmatrix} = \begin{bmatrix} x_1 & x_1 & \cdots & x_1 \\ x_2 & x_2 & \cdots & x_2 \\ \vdots & \vdots & \ddots & \vdots \\ x_m & x_m & \cdots & x_m \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \begin{bmatrix} 1 & 1 & \cdots & 1 \end{bmatrix} = x \mathbf{1}^T.$$

2.2 Matrix-Vector Products

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $x \in \mathbb{R}^n$, their product is a vector $y = Ax \in \mathbb{R}^m$. There are a couple ways of looking at matrix-vector multiplication, and we will look at each of them in turn.

If we write A by rows, then we can express Ax as,

$$y = Ax = \begin{bmatrix} \text{---} & a_1^T & \text{---} \\ \text{---} & a_2^T & \text{---} \\ & \vdots & \\ \text{---} & a_m^T & \text{---} \end{bmatrix} x = \begin{bmatrix} a_1^T x \\ a_2^T x \\ \vdots \\ a_m^T x \end{bmatrix}.$$

In other words, the i th entry of y is equal to the inner product of the i th *row* of A and x , $y_i = a_i^T x$.

Alternatively, let's write A in column form. In this case we see that,

$$y = Ax = \begin{bmatrix} | & | & \cdots & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & \cdots & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_1 \end{bmatrix} x_1 + \begin{bmatrix} a_2 \end{bmatrix} x_2 + \cdots + \begin{bmatrix} a_n \end{bmatrix} x_n .$$

In other words, y is a **linear combination** of the *columns* of A , where the coefficients of the linear combination are given by the entries of x .

So far we have been multiplying on the right by a column vector, but it is also possible to multiply on the left by a row vector. This is written, $y^T = x^T A$ for $A \in \mathbb{R}^{m \times n}$, $x \in \mathbb{R}^m$, and $y \in \mathbb{R}^n$. As before, we can express y^T in two obvious ways, depending on whether we express A in terms of its rows or columns. In the first case we express A in terms of its columns, which gives

$$y^T = x^T A = x^T \begin{bmatrix} | & | & \cdots & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & \cdots & | \end{bmatrix} = [x^T a_1 \quad x^T a_2 \quad \cdots \quad x^T a_n]$$

which demonstrates that the i th entry of y^T is equal to the inner product of x and the i th *column* of A .

Finally, expressing A in terms of rows we get the final representation of the vector-matrix product,

$$\begin{aligned} y^T &= x^T A \\ &= [x_1 \quad x_2 \quad \cdots \quad x_n] \begin{bmatrix} - & a_1^T & - \\ - & a_2^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix} \\ &= x_1 [- \quad a_1^T \quad -] + x_2 [- \quad a_2^T \quad -] + \cdots + x_n [- \quad a_n^T \quad -] \end{aligned}$$

so we see that y^T is a linear combination of the *rows* of A , where the coefficients for the linear combination are given by the entries of x .

2.3 Matrix-Matrix Products

Armed with this knowledge, we can now look at four different (but, of course, equivalent) ways of viewing the matrix-matrix multiplication $C = AB$ as defined at the beginning of this section.

First, we can view matrix-matrix multiplication as a set of vector-vector products. The most obvious viewpoint, which follows immediately from the definition, is that the (i, j) th

entry of C is equal to the inner product of the i th row of A and the j th column of B . Symbolically, this looks like the following,

$$C = AB = \begin{bmatrix} - & a_1^T & - \\ - & a_2^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix} \begin{bmatrix} | & | & & | \\ b_1 & b_2 & \cdots & b_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} a_1^T b_1 & a_1^T b_2 & \cdots & a_1^T b_p \\ a_2^T b_1 & a_2^T b_2 & \cdots & a_2^T b_p \\ \vdots & \vdots & \ddots & \vdots \\ a_m^T b_1 & a_m^T b_2 & \cdots & a_m^T b_p \end{bmatrix}.$$

Remember that since $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, $a_i \in \mathbb{R}^n$ and $b_j \in \mathbb{R}^n$, so these inner products all make sense. This is the most “natural” representation when we represent A by rows and B by columns. Alternatively, we can represent A by columns, and B by rows. This representation leads to a much trickier interpretation of AB as a sum of outer products. Symbolically,

$$C = AB = \begin{bmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} - & b_1^T & - \\ - & b_2^T & - \\ & \vdots & \\ - & b_n^T & - \end{bmatrix} = \sum_{i=1}^n a_i b_i^T.$$

Put another way, AB is equal to the sum, over all i , of the outer product of the i th column of A and the i th row of B . Since, in this case, $a_i \in \mathbb{R}^m$ and $b_i \in \mathbb{R}^p$, the dimension of the outer product $a_i b_i^T$ is $m \times p$, which coincides with the dimension of C . Chances are, the last equality above may appear confusing to you. If so, take the time to check it for yourself!

Second, we can also view matrix-matrix multiplication as a set of matrix-vector products. Specifically, if we represent B by columns, we can view the columns of C as matrix-vector products between A and the columns of B . Symbolically,

$$C = AB = A \begin{bmatrix} | & | & & | \\ b_1 & b_2 & \cdots & b_p \\ | & | & & | \end{bmatrix} = \begin{bmatrix} | & | & & | \\ Ab_1 & Ab_2 & \cdots & Ab_p \\ | & | & & | \end{bmatrix}.$$

Here the i th column of C is given by the matrix-vector product with the vector on the right, $c_i = Ab_i$. These matrix-vector products can in turn be interpreted using both viewpoints given in the previous subsection. Finally, we have the analogous viewpoint, where we represent A by rows, and view the rows of C as the matrix-vector product between the rows of A and C . Symbolically,

$$C = AB = \begin{bmatrix} - & a_1^T & - \\ - & a_2^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix} B = \begin{bmatrix} - & a_1^T B & - \\ - & a_2^T B & - \\ & \vdots & \\ - & a_m^T B & - \end{bmatrix}.$$

Here the i th row of C is given by the matrix-vector product with the vector on the left, $c_i^T = a_i^T B$.

It may seem like overkill to dissect matrix multiplication to such a large degree, especially when all these viewpoints follow immediately from the initial definition we gave (in about a line of math) at the beginning of this section. However, virtually all of linear algebra deals with matrix multiplications of some kind, and it is worthwhile to spend some time trying to develop an intuitive understanding of the viewpoints presented here.

In addition to this, it is useful to know a few basic properties of matrix multiplication at a higher level:

- Matrix multiplication is associative: $(AB)C = A(BC)$.
- Matrix multiplication is distributive: $A(B + C) = AB + AC$.
- Matrix multiplication is, in general, *not* commutative; that is, it can be the case that $AB \neq BA$. (For example, if $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times q}$, the matrix product BA does not even exist if m and q are not equal!)

If you are not familiar with these properties, take the time to verify them for yourself. For example, to check the associativity of matrix multiplication, suppose that $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, and $C \in \mathbb{R}^{p \times q}$. Note that $AB \in \mathbb{R}^{m \times p}$, so $(AB)C \in \mathbb{R}^{m \times q}$. Similarly, $BC \in \mathbb{R}^{n \times q}$, so $A(BC) \in \mathbb{R}^{m \times q}$. Thus, the dimensions of the resulting matrices agree. To show that matrix multiplication is associative, it suffices to check that the (i, j) th entry of $(AB)C$ is equal to the (i, j) th entry of $A(BC)$. We can verify this directly using the definition of matrix multiplication:

$$\begin{aligned} ((AB)C)_{ij} &= \sum_{k=1}^p (AB)_{ik} C_{kj} = \sum_{k=1}^p \left(\sum_{l=1}^n A_{il} B_{lk} \right) C_{kj} \\ &= \sum_{k=1}^p \left(\sum_{l=1}^n A_{il} B_{lk} C_{kj} \right) = \sum_{l=1}^n \left(\sum_{k=1}^p A_{il} B_{lk} C_{kj} \right) \\ &= \sum_{l=1}^n A_{il} \left(\sum_{k=1}^p B_{lk} C_{kj} \right) = \sum_{l=1}^n A_{il} (BC)_{lj} = (A(BC))_{ij}. \end{aligned}$$

Here, the first and last two equalities simply use the definition of matrix multiplication, the third and fifth equalities use the distributive property for *scalar multiplication over addition*, and the fourth equality uses the *commutative and associativity of scalar addition*. This technique for proving matrix properties by reduction to simple scalar properties will come up often, so make sure you're familiar with it.

3 Operations and Properties

In this section we present several operations and properties of matrices and vectors. Hopefully a great deal of this will be review for you, so the notes can just serve as a reference for these topics.

3.1 The Identity Matrix and Diagonal Matrices

The **identity matrix**, denoted $I \in \mathbb{R}^{n \times n}$, is a square matrix with ones on the diagonal and zeros everywhere else. That is,

$$I_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

It has the property that for all $A \in \mathbb{R}^{m \times n}$,

$$AI = A = IA.$$

Note that in some sense, the notation for the identity matrix is ambiguous, since it does not specify the dimension of I . Generally, the dimensions of I are inferred from context so as to make matrix multiplication possible. For example, in the equation above, the I in $AI = A$ is an $n \times n$ matrix, whereas the I in $A = IA$ is an $m \times m$ matrix.

A **diagonal matrix** is a matrix where all non-diagonal elements are 0. This is typically denoted $D = \text{diag}(d_1, d_2, \dots, d_n)$, with

$$D_{ij} = \begin{cases} d_i & i = j \\ 0 & i \neq j \end{cases}$$

Clearly, $I = \text{diag}(1, 1, \dots, 1)$.

3.2 The Transpose

The **transpose** of a matrix results from “flipping” the rows and columns. Given a matrix $A \in \mathbb{R}^{m \times n}$, its transpose, written $A^T \in \mathbb{R}^{n \times m}$, is the $n \times m$ matrix whose entries are given by

$$(A^T)_{ij} = A_{ji}.$$

We have in fact already been using the transpose when describing row vectors, since the transpose of a column vector is naturally a row vector.

The following properties of transposes are easily verified:

- $(A^T)^T = A$
- $(AB)^T = B^T A^T$
- $(A + B)^T = A^T + B^T$

3.3 Symmetric Matrices

A square matrix $A \in \mathbb{R}^{n \times n}$ is **symmetric** if $A = A^T$. It is **anti-symmetric** if $A = -A^T$. It is easy to show that for any matrix $A \in \mathbb{R}^{n \times n}$, the matrix $A + A^T$ is symmetric and the

matrix $A - A^T$ is anti-symmetric. From this it follows that any square matrix $A \in \mathbb{R}^{n \times n}$ can be represented as a sum of a symmetric matrix and an anti-symmetric matrix, since

$$A = \frac{1}{2}(A + A^T) + \frac{1}{2}(A - A^T)$$

and the first matrix on the right is symmetric, while the second is anti-symmetric. It turns out that symmetric matrices occur a great deal in practice, and they have many nice properties which we will look at shortly. It is common to denote the set of all symmetric matrices of size n as $\mathbb{S}^n$, so that $A \in \mathbb{S}^n$ means that A is a symmetric $n \times n$ matrix;

3.4 The Trace

The **trace** of a square matrix $A \in \mathbb{R}^{n \times n}$, denoted $\text{tr}(A)$ (or just $\text{tr}A$ if the parentheses are obviously implied), is the sum of diagonal elements in the matrix:

$$\text{tr}A = \sum_{i=1}^n A_{ii}.$$

As described in the CS229 lecture notes, the trace has the following properties (included here for the sake of completeness):

- For $A \in \mathbb{R}^{n \times n}$, $\text{tr}A = \text{tr}A^T$.
- For $A, B \in \mathbb{R}^{n \times n}$, $\text{tr}(A + B) = \text{tr}A + \text{tr}B$.
- For $A \in \mathbb{R}^{n \times n}$, $t \in \mathbb{R}$, $\text{tr}(tA) = t \text{tr}A$.
- For A, B such that AB is square, $\text{tr}AB = \text{tr}BA$.
- For A, B, C such that ABC is square, $\text{tr}ABC = \text{tr}BCA = \text{tr}CAB$, and so on for the product of more matrices.

As an example of how these properties can be proven, we'll consider the fourth property given above. Suppose that $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times m}$ (so that $AB \in \mathbb{R}^{m \times m}$ is a square matrix). Observe that $BA \in \mathbb{R}^{n \times n}$ is also a square matrix, so it makes sense to apply the trace operator to it. To verify that $\text{tr}AB = \text{tr}BA$, note that

$$\begin{aligned} \text{tr}AB &= \sum_{i=1}^m (AB)_{ii} = \sum_{i=1}^m \left(\sum_{j=1}^n A_{ij} B_{ji} \right) \\ &= \sum_{i=1}^m \sum_{j=1}^n A_{ij} B_{ji} = \sum_{j=1}^n \sum_{i=1}^m B_{ji} A_{ij} \\ &= \sum_{j=1}^n \left(\sum_{i=1}^m B_{ji} A_{ij} \right) = \sum_{j=1}^n (BA)_{jj} = \text{tr}BA. \end{aligned}$$

Here, the first and last two equalities use the definition of the trace operator and matrix multiplication. The fourth equality, where the main work occurs, uses the commutativity of scalar multiplication in order to reverse the order of the terms in each product, and the commutativity and associativity of scalar addition in order to rearrange the order of the summation.

3.5 Norms

A **norm** of a vector $\|x\|$ is informally a measure of the “length” of the vector. For example, we have the commonly-used Euclidean or ℓ_2 norm,

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}.$$

Note that $\|x\|_2^2 = x^T x$.

More formally, a norm is any function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ that satisfies 4 properties:

1. For all $x \in \mathbb{R}^n$, $f(x) \geq 0$ (non-negativity).
2. $f(x) = 0$ if and only if $x = 0$ (definiteness).
3. For all $x \in \mathbb{R}^n$, $t \in \mathbb{R}$, $f(tx) = |t|f(x)$ (homogeneity).
4. For all $x, y \in \mathbb{R}^n$, $f(x + y) \leq f(x) + f(y)$ (triangle inequality).

Other examples of norms are the ℓ_1 norm,

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

and the ℓ_∞ norm,

$$\|x\|_\infty = \max_i |x_i|.$$

In fact, all three norms presented so far are examples of the family of ℓ_p norms, which are parameterized by a real number $p \geq 1$, and defined as

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

Norms can also be defined for matrices, such as the Frobenius norm,

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n A_{ij}^2} = \sqrt{\text{tr}(A^T A)}.$$

Many other norms exist, but they are beyond the scope of this review.

3.6 Linear Independence and Rank

A set of vectors $\{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^m$ is said to be **(linearly) independent** if no vector can be represented as a linear combination of the remaining vectors. Conversely, if one vector belonging to the set *can* be represented as a linear combination of the remaining vectors, then the vectors are said to be **(linearly) dependent**. That is, if

$$x_n = \sum_{i=1}^{n-1} \alpha_i x_i$$

for some scalar values $\alpha_1, \dots, \alpha_{n-1} \in \mathbb{R}$, then we say that the vectors $x_1, \dots, x_n$ are linearly dependent; otherwise, the vectors are linearly independent. For example, the vectors

$$x_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad x_2 = \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix} \quad x_3 = \begin{bmatrix} 2 \\ -3 \\ -1 \end{bmatrix}$$

are linearly dependent because $x_3 = -2x_1 + x_2$.

The **column rank** of a matrix $A \in \mathbb{R}^{m \times n}$ is the size of the largest subset of columns of A that constitute a linearly independent set. With some abuse of terminology, this is often referred to simply as the number of linearly independent columns of A . In the same way, the **row rank** is the largest number of rows of A that constitute a linearly independent set.

For any matrix $A \in \mathbb{R}^{m \times n}$, it turns out that the column rank of A is equal to the row rank of A (though we will not prove this), and so both quantities are referred to collectively as the **rank** of A , denoted as $\text{rank}(A)$. The following are some basic properties of the rank:

- For $A \in \mathbb{R}^{m \times n}$, $\text{rank}(A) \leq \min(m, n)$. If $\text{rank}(A) = \min(m, n)$, then A is said to be **full rank**.
- For $A \in \mathbb{R}^{m \times n}$, $\text{rank}(A) = \text{rank}(A^T)$.
- For $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$.
- For $A, B \in \mathbb{R}^{m \times n}$, $\text{rank}(A + B) \leq \text{rank}(A) + \text{rank}(B)$.

3.7 The Inverse

The **inverse** of a square matrix $A \in \mathbb{R}^{n \times n}$ is denoted A^{-1} , and is the unique matrix such that

$$A^{-1}A = I = AA^{-1}.$$

Note that not all matrices have inverses. Non-square matrices, for example, do not have inverses by definition. However, for some square matrices A , it may still be the case that

A^{-1} may not exist. In particular, we say that A is *invertible* or *non-singular* if A^{-1} exists and *non-invertible* or *singular* otherwise.¹

In order for a square matrix A to have an inverse A^{-1} , then A must be full rank. We will soon see that there are many alternative sufficient and necessary conditions, in addition to full rank, for invertibility.

The following are properties of the inverse; all assume that $A, B \in \mathbb{R}^{n \times n}$ are non-singular:

- $(A^{-1})^{-1} = A$
- $(AB)^{-1} = B^{-1}A^{-1}$
- $(A^{-1})^T = (A^T)^{-1}$. For this reason this matrix is often denoted A^{-T} .

As an example of how the inverse is used, consider the linear system of equations, $Ax = b$ where $A \in \mathbb{R}^{n \times n}$, and $x, b \in \mathbb{R}^n$. If A is nonsingular (i.e., invertible), then $x = A^{-1}b$. (What if $A \in \mathbb{R}^{m \times n}$ is not a square matrix? Does this work?)

3.8 Orthogonal Matrices

Two vectors $x, y \in \mathbb{R}^n$ are *orthogonal* if $x^T y = 0$. A vector $x \in \mathbb{R}^n$ is *normalized* if $\|x\|_2 = 1$. A square matrix $U \in \mathbb{R}^{n \times n}$ is *orthogonal* (note the different meanings when talking about vectors versus matrices) if all its columns are orthogonal to each other and are normalized (the columns are then referred to as being *orthonormal*).

It follows immediately from the definition of orthogonality and normality that

$$U^T U = I = U U^T.$$

In other words, the inverse of an orthogonal matrix is its transpose. Note that if U is not square — i.e., $U \in \mathbb{R}^{m \times n}$, $n < m$ — but its columns are still orthonormal, then $U^T U = I$, but $U U^T \neq I$. We generally only use the term orthogonal to describe the previous case, where U is square.

Another nice property of orthogonal matrices is that operating on a vector with an orthogonal matrix will not change its Euclidean norm, i.e.,

$$\|Ux\|_2 = \|x\|_2$$

for any $x \in \mathbb{R}^n$, $U \in \mathbb{R}^{n \times n}$ orthogonal.

3.9 Range and Nullspace of a Matrix

The *span* of a set of vectors $\{x_1, x_2, \dots, x_n\}$ is the set of all vectors that can be expressed as a linear combination of $\{x_1, \dots, x_n\}$. That is,

$$\text{span}(\{x_1, \dots, x_n\}) = \left\{ v : v = \sum_{i=1}^n \alpha_i x_i, \alpha_i \in \mathbb{R} \right\}.$$

¹It's easy to get confused and think that non-singular means non-invertible. But in fact, it means the opposite! Watch out!

It can be shown that if $\{x_1, \dots, x_n\}$ is a set of n linearly independent vectors, where each $x_i \in \mathbb{R}^n$, then $\text{span}(\{x_1, \dots, x_n\}) = \mathbb{R}^n$. In other words, *any* vector $v \in \mathbb{R}^n$ can be written as a linear combination of x_1 through x_n . The **projection** of a vector $y \in \mathbb{R}^m$ onto the span of $\{x_1, \dots, x_n\}$ (here we assume $x_i \in \mathbb{R}^m$) is the vector $v \in \text{span}(\{x_1, \dots, x_n\})$, such that v is as close as possible to y , as measured by the Euclidean norm $\|v - y\|_2$. We denote the projection as $\text{Proj}(y; \{x_1, \dots, x_n\})$ and can define it formally as,

$$\text{Proj}(y; \{x_1, \dots, x_n\}) = \text{argmin}_{v \in \text{span}(\{x_1, \dots, x_n\})} \|y - v\|_2.$$

The **range** (sometimes also called the columnspace) of a matrix $A \in \mathbb{R}^{m \times n}$, denoted $\mathcal{R}(A)$, is the the span of the columns of A . In other words,

$$\mathcal{R}(A) = \{v \in \mathbb{R}^m : v = Ax, x \in \mathbb{R}^n\}.$$

Making a few technical assumptions (namely that A is full rank and that $n < m$), the projection of a vector $y \in \mathbb{R}^m$ onto the range of A is given by,

$$\text{Proj}(y; A) = \text{argmin}_{v \in \mathcal{R}(A)} \|v - y\|_2 = A(A^T A)^{-1} A^T y .$$

This last equation should look extremely familiar, since it is almost the same formula we derived in class (and which we will soon derive again) for the least squares estimation of parameters. Looking at the definition for the projection, it should not be too hard to convince yourself that this is in fact the same objective that we minimized in our least squares problem (except for a squaring of the norm, which doesn't affect the optimal point) and so these problems are naturally very connected. When A contains only a single column, $a \in \mathbb{R}^m$, this gives the special case for a projection of a vector on to a line:

$$\text{Proj}(y; a) = \frac{aa^T}{a^T a} y .$$

The **nullspace** of a matrix $A \in \mathbb{R}^{m \times n}$, denoted $\mathcal{N}(A)$ is the set of all vectors that equal 0 when multiplied by A , i.e.,

$$\mathcal{N}(A) = \{x \in \mathbb{R}^n : Ax = 0\}.$$

Note that vectors in $\mathcal{R}(A)$ are of size m , while vectors in the $\mathcal{N}(A)$ are of size n , so vectors in $\mathcal{R}(A^T)$ and $\mathcal{N}(A)$ are both in $\mathbb{R}^n$. In fact, we can say much more. It turns out that

$$\{w : w = u + v, u \in \mathcal{R}(A^T), v \in \mathcal{N}(A)\} = \mathbb{R}^n \text{ and } \mathcal{R}(A^T) \cap \mathcal{N}(A) = \{\mathbf{0}\} .$$

In other words, $\mathcal{R}(A^T)$ and $\mathcal{N}(A)$ are disjoint subsets that together span the entire space of $\mathbb{R}^n$. Sets of this type are called **orthogonal complements**, and we denote this $\mathcal{R}(A^T) = \mathcal{N}(A)^\perp$.

3.10 The Determinant

The **determinant** of a square matrix $A \in \mathbb{R}^{n \times n}$, is a function $\det : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}$, and is denoted $|A|$ or $\det A$ (like the trace operator, we usually omit parentheses). Algebraically, one could write down an explicit formula for the determinant of A , but this unfortunately gives little intuition about its meaning. Instead, we'll start out by providing a geometric interpretation of the determinant and then visit some of its specific algebraic properties afterwards.

Given a matrix

$$\begin{bmatrix} - & a_1^T & - \\ - & a_2^T & - \\ & \vdots & \\ - & a_n^T & - \end{bmatrix},$$

consider the set of points $S \subset \mathbb{R}^n$ formed by taking all possible linear combinations of the row vectors $a_1, \dots, a_n \in \mathbb{R}^n$ of A , where the coefficients of the linear combination are all between 0 and 1; that is, the set S is the restriction of $\text{span}(\{a_1, \dots, a_n\})$ to only those linear combinations whose coefficients $\alpha_1, \dots, \alpha_n$ satisfy $0 \leq \alpha_i \leq 1, i = 1, \dots, n$. Formally,

$$S = \{v \in \mathbb{R}^n : v = \sum_{i=1}^n \alpha_i a_i \text{ where } 0 \leq \alpha_i \leq 1, i = 1, \dots, n\}.$$

The absolute value of the determinant of A , it turns out, is a measure of the “volume” of the set S .²

For example, consider the 2×2 matrix,

$$A = \begin{bmatrix} 1 & 3 \\ 3 & 2 \end{bmatrix}. \tag{1}$$

Here, the rows of the matrix are

$$a_1 = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \quad a_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

The set S corresponding to these rows is shown in Figure 1. For two-dimensional matrices, S generally has the shape of a *parallelogram*. In our example, the value of the determinant is $|A| = -7$ (as can be computed using the formulas shown later in this section), so the area of the parallelogram is 7. (Verify this for yourself!)

In three dimensions, the set S corresponds to an object known as a *parallelepiped* (a three-dimensional box with skewed sides, such that every face has the shape of a parallelogram). The absolute value of the determinant of the 3×3 matrix whose rows define S give the three-dimensional volume of the parallelepiped. In even higher dimensions, the set S is an object known as an n -dimensional *parallelotope*.

²Admittedly, we have not actually defined what we mean by “volume” here, but hopefully the intuition should be clear enough. When $n = 2$, our notion of “volume” corresponds to the area of S in the Cartesian plane. When $n = 3$, “volume” corresponds with our usual notion of volume for a three-dimensional object.

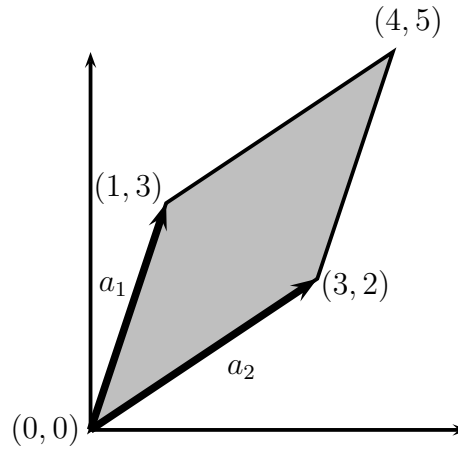


Figure 1: Illustration of the determinant for the 2×2 matrix A given in (1). Here, a_1 and a_2 are vectors corresponding to the rows of A , and the set S corresponds to the shaded region (i.e., the parallelogram). The absolute value of the determinant, $|\det A| = 7$, is the area of the parallelogram.

Algebraically, the determinant satisfies the following three properties (from which all other properties follow, including the general formula):

1. The determinant of the identity is 1, $|I| = 1$. (Geometrically, the volume of a unit hypercube is 1).
2. Given a matrix $A \in \mathbb{R}^{n \times n}$, if we multiply a single row in A by a scalar $t \in \mathbb{R}$, then the determinant of the new matrix is $t|A|$,

$$\left| \begin{bmatrix} - & t a_1^T & - \\ - & a_2^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix} \right| = t|A|.$$

(Geometrically, multiplying one of the sides of the set S by a factor t causes the volume to increase by a factor t .)

3. If we exchange any two rows a_i^T and a_j^T of A , then the determinant of the new matrix is $-|A|$, for example

$$\left| \begin{bmatrix} - & a_2^T & - \\ - & a_1^T & - \\ & \vdots & \\ - & a_m^T & - \end{bmatrix} \right| = -|A|.$$

In case you are wondering, it is not immediately obvious that a function satisfying the above three properties exists. In fact, though, such a function does exist, and is unique (which we will not prove here).

Several properties that follow from the three properties above include:

- For $A \in \mathbb{R}^{n \times n}$, $|A| = |A^T|$.
- For $A, B \in \mathbb{R}^{n \times n}$, $|AB| = |A||B|$.
- For $A \in \mathbb{R}^{n \times n}$, $|A| = 0$ if and only if A is singular (i.e., non-invertible). (If A is singular then it does not have full rank, and hence its columns are linearly dependent. In this case, the set S corresponds to a “flat sheet” within the n -dimensional space and hence has zero volume.)
- For $A \in \mathbb{R}^{n \times n}$ and A non-singular, $|A^{-1}| = 1/|A|$.

Before giving the general definition for the determinant, we define, for $A \in \mathbb{R}^{n \times n}$, $A_{\setminus i, \setminus j} \in \mathbb{R}^{(n-1) \times (n-1)}$ to be the *matrix* that results from deleting the i th row and j th column from A . The general (recursive) formula for the determinant is

$$\begin{aligned} |A| &= \sum_{i=1}^n (-1)^{i+j} a_{ij} |A_{\setminus i, \setminus j}| \quad (\text{for any } j \in 1, \dots, n) \\ &= \sum_{j=1}^n (-1)^{i+j} a_{ij} |A_{\setminus i, \setminus j}| \quad (\text{for any } i \in 1, \dots, n) \end{aligned}$$

with the initial case that $|A| = a_{11}$ for $A \in \mathbb{R}^{1 \times 1}$. If we were to expand this formula completely for $A \in \mathbb{R}^{n \times n}$, there would be a total of $n!$ (n factorial) different terms. For this reason, we hardly ever explicitly write the complete equation of the determinant for matrices bigger than 3×3 . However, the equations for determinants of matrices up to size 3×3 are fairly common, and it is good to know them:

$$\begin{aligned} |[a_{11}]| &= a_{11} \\ \left| \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \right| &= a_{11}a_{22} - a_{12}a_{21} \\ \left| \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \right| &= a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} \\ &\quad - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33} - a_{13}a_{22}a_{31} \end{aligned}$$

The **classical adjoint** (often just called the adjoint) of a matrix $A \in \mathbb{R}^{n \times n}$, is denoted $\text{adj}(A)$, and defined as

$$\text{adj}(A) \in \mathbb{R}^{n \times n}, \quad (\text{adj}(A))_{ij} = (-1)^{i+j} |A_{\setminus j, \setminus i}|$$

(note the switch in the indices $A_{\setminus j, \setminus i}$). It can be shown that for any nonsingular $A \in \mathbb{R}^{n \times n}$,

$$A^{-1} = \frac{1}{|A|} \text{adj}(A) .$$

While this is a nice “explicit” formula for the inverse of matrix, we should note that, numerically, there are in fact much more efficient ways of computing the inverse.

3.11 Quadratic Forms and Positive Semidefinite Matrices

Given a square matrix $A \in \mathbb{R}^{n \times n}$ and a vector $x \in \mathbb{R}^n$, the scalar value $x^T A x$ is called a **quadratic form**. Written explicitly, we see that

$$x^T A x = \sum_{i=1}^n x_i (A x)_i = \sum_{i=1}^n x_i \left(\sum_{j=1}^n A_{ij} x_j \right) = \sum_{i=1}^n \sum_{j=1}^n A_{ij} x_i x_j .$$

Note that,

$$x^T A x = (x^T A x)^T = x^T A^T x = x^T \left(\frac{1}{2} A + \frac{1}{2} A^T \right) x,$$

where the first equality follows from the fact that the transpose of a scalar is equal to itself, and the second equality follows from the fact that we are averaging two quantities which are themselves equal. From this, we can conclude that only the symmetric part of A contributes to the quadratic form. For this reason, we often implicitly assume that the matrices appearing in a quadratic form are symmetric.

We give the following definitions:

- A symmetric matrix $A \in \mathbb{S}^n$ is **positive definite** (PD) if for all non-zero vectors $x \in \mathbb{R}^n$, $x^T A x > 0$. This is usually denoted $A \succ 0$ (or just $A > 0$), and often times the set of all positive definite matrices is denoted $\mathbb{S}_{++}^n$.
- A symmetric matrix $A \in \mathbb{S}^n$ is **positive semidefinite** (PSD) if for all vectors $x \in \mathbb{R}^n$, $x^T A x \geq 0$. This is written $A \succeq 0$ (or just $A \geq 0$), and the set of all positive semidefinite matrices is often denoted $\mathbb{S}_+^n$.
- Likewise, a symmetric matrix $A \in \mathbb{S}^n$ is **negative definite** (ND), denoted $A \prec 0$ (or just $A < 0$) if for all non-zero $x \in \mathbb{R}^n$, $x^T A x < 0$.
- Similarly, a symmetric matrix $A \in \mathbb{S}^n$ is **negative semidefinite** (NSD), denoted $A \preceq 0$ (or just $A \leq 0$) if for all $x \in \mathbb{R}^n$, $x^T A x \leq 0$.
- Finally, a symmetric matrix $A \in \mathbb{S}^n$ is **indefinite**, if it is neither positive semidefinite nor negative semidefinite — i.e., if there exists $x_1, x_2 \in \mathbb{R}^n$ such that $x_1^T A x_1 > 0$ and $x_2^T A x_2 < 0$.

It should be obvious that if A is positive definite, then $-A$ is negative definite and vice versa. Likewise, if A is positive semidefinite then $-A$ is negative semidefinite and vice versa. If A is indefinite, then so is $-A$.

One important property of positive definite and negative definite matrices is that they are always full rank, and hence, invertible. To see why this is the case, suppose that some matrix $A \in \mathbb{R}^{n \times n}$ is not full rank. Then, suppose that the j th column of A is expressible as a linear combination of other $n - 1$ columns:

$$a_j = \sum_{i \neq j} x_i a_i,$$

for some $x_1, \dots, x_{j-1}, x_{j+1}, \dots, x_n \in \mathbb{R}$. Setting $x_j = -1$, we have

$$Ax = \sum_{i=1}^n x_i a_i = 0.$$

But this implies $x^T Ax = 0$ for some non-zero vector x , so A must be neither positive definite nor negative definite. Therefore, if A is either positive definite or negative definite, it must be full rank.

Finally, there is one type of positive definite matrix that comes up frequently, and so deserves some special mention. Given any matrix $A \in \mathbb{R}^{m \times n}$ (not necessarily symmetric or even square), the matrix $G = A^T A$ (sometimes called a ***Gram matrix***) is always positive semidefinite. Further, if $m \geq n$ (and we assume for convenience that A is full rank), then $G = A^T A$ is positive definite.

3.12 Eigenvalues and Eigenvectors

Given a square matrix $A \in \mathbb{R}^{n \times n}$, we say that $\lambda \in \mathbb{C}$ is an ***eigenvalue*** of A and $x \in \mathbb{C}^n$ is the corresponding ***eigenvector***³ if

$$Ax = \lambda x, \quad x \neq 0.$$

Intuitively, this definition means that multiplying A by the vector x results in a new vector that points in the same direction as x , but scaled by a factor λ . Also note that for any eigenvector $x \in \mathbb{C}^n$, and scalar $t \in \mathbb{C}$, $A(cx) = cAx = c\lambda x = \lambda(cx)$, so cx is also an eigenvector. For this reason when we talk about “the” eigenvector associated with λ , we usually assume that the eigenvector is normalized to have length 1 (this still creates some ambiguity, since x and $-x$ will both be eigenvectors, but we will have to live with this).

We can rewrite the equation above to state that (λ, x) is an eigenvalue-eigenvector pair of A if,

$$(\lambda I - A)x = 0, \quad x \neq 0.$$

But $(\lambda I - A)x = 0$ has a non-zero solution to x if and only if $(\lambda I - A)$ has a non-empty nullspace, which is only the case if $(\lambda I - A)$ is singular, i.e.,

$$|(\lambda I - A)| = 0.$$

We can now use the previous definition of the determinant to expand this expression into a (very large) polynomial in λ , where λ will have maximum degree n . We then find the n (possibly complex) roots of this polynomial to find the n eigenvalues $\lambda_1, \dots, \lambda_n$. To find the eigenvector corresponding to the eigenvalue λ_i , we simply solve the linear equation $(\lambda_i I - A)x = 0$. It should be noted that this is not the method which is actually used

³Note that λ and the entries of x are actually in $\mathbb{C}$, the set of complex numbers, not just the reals; we will see shortly why this is necessary. Don't worry about this technicality for now, you can think of complex vectors in the same way as real vectors.

in practice to numerically compute the eigenvalues and eigenvectors (remember that the complete expansion of the determinant has $n!$ terms); it is rather a mathematical argument.

The following are properties of eigenvalues and eigenvectors (in all cases assume $A \in \mathbb{R}^{n \times n}$ has eigenvalues $\lambda_1, \dots, \lambda_n$ and associated eigenvectors $x_1, \dots, x_n$):

- The trace of a A is equal to the sum of its eigenvalues,

$$\text{tr} A = \sum_{i=1}^n \lambda_i.$$

- The determinant of A is equal to the product of its eigenvalues,

$$|A| = \prod_{i=1}^n \lambda_i.$$

- The rank of A is equal to the number of non-zero eigenvalues of A .
- If A is non-singular then $1/\lambda_i$ is an eigenvalue of A^{-1} with associated eigenvector x_i , i.e., $A^{-1}x_i = (1/\lambda_i)x_i$. (To prove this, take the eigenvector equation, $Ax_i = \lambda_i x_i$ and left-multiply each side by A^{-1} .)
- The eigenvalues of a diagonal matrix $D = \text{diag}(d_1, \dots, d_n)$ are just the diagonal entries $d_1, \dots, d_n$.

We can write all the eigenvector equations simultaneously as

$$AX = X\Lambda$$

where the columns of $X \in \mathbb{R}^{n \times n}$ are the eigenvectors of A and Λ is a diagonal matrix whose entries are the eigenvalues of A , i.e.,

$$X \in \mathbb{R}^{n \times n} = \begin{bmatrix} | & | & \cdots & | \\ x_1 & x_2 & \cdots & x_n \\ | & | & \cdots & | \end{bmatrix}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n).$$

If the eigenvectors of A are linearly independent, then the matrix X will be invertible, so $A = X\Lambda X^{-1}$. A matrix that can be written in this form is called **diagonalizable**.

3.13 Eigenvalues and Eigenvectors of Symmetric Matrices

Two remarkable properties come about when we look at the eigenvalues and eigenvectors of a symmetric matrix $A \in \mathbb{S}^n$. First, it can be shown that all the eigenvalues of A are real. Secondly, the eigenvectors of A are orthonormal, i.e., the matrix X defined above is an orthogonal matrix (for this reason, we denote the matrix of eigenvectors as U in this case).

We can therefore represent A as $A = U\Lambda U^T$, remembering from above that the inverse of an orthogonal matrix is just its transpose.

Using this, we can show that the definiteness of a matrix depends entirely on the sign of its eigenvalues. Suppose $A \in \mathbb{S}^n = U\Lambda U^T$. Then

$$x^T A x = x^T U \Lambda U^T x = y^T \Lambda y = \sum_{i=1}^n \lambda_i y_i^2$$

where $y = U^T x$ (and since U is full rank, any vector $y \in \mathbb{R}^n$ can be represented in this form). Because y_i^2 is always positive, the sign of this expression depends entirely on the λ_i 's. If all $\lambda_i > 0$, then the matrix is positive definite; if all $\lambda_i \geq 0$, it is positive semidefinite. Likewise, if all $\lambda_i < 0$ or $\lambda_i \leq 0$, then A is negative definite or negative semidefinite respectively. Finally, if A has both positive and negative eigenvalues, it is indefinite.

An application where eigenvalues and eigenvectors come up frequently is in maximizing some function of a matrix. In particular, for a matrix $A \in \mathbb{S}^n$, consider the following maximization problem,

$$\max_{x \in \mathbb{R}^n} x^T A x \quad \text{subject to } \|x\|_2^2 = 1$$

i.e., we want to find the vector (of norm 1) which maximizes the quadratic form. Assuming the eigenvalues are ordered as $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$, the optimal x for this optimization problem is x_1 , the eigenvector corresponding to λ_1 . In this case the maximal value of the quadratic form is λ_1 . Similarly, the optimal solution to the minimization problem,

$$\min_{x \in \mathbb{R}^n} x^T A x \quad \text{subject to } \|x\|_2^2 = 1$$

is x_n , the eigenvector corresponding to λ_n , and the minimal value is λ_n . This can be proved by appealing to the eigenvector-eigenvalue form of A and the properties of orthogonal matrices. However, in the next section we will see a way of showing it directly using matrix calculus.

4 Matrix Calculus

While the topics in the previous sections are typically covered in a standard course on linear algebra, one topic that does not seem to be covered very often (and which we will use extensively) is the extension of calculus to the vector setting. Despite the fact that all the actual calculus we use is relatively trivial, the notation can often make things look much more difficult than they are. In this section we present some basic definitions of matrix calculus and provide a few examples.

4.1 The Gradient

Suppose that $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ is a function that takes as input a matrix A of size $m \times n$ and returns a real value. Then the **gradient** of f (with respect to $A \in \mathbb{R}^{m \times n}$) is the matrix of

partial derivatives, defined as:

$$\nabla_A f(A) \in \mathbb{R}^{m \times n} = \begin{bmatrix} \frac{\partial f(A)}{\partial A_{11}} & \frac{\partial f(A)}{\partial A_{12}} & \dots & \frac{\partial f(A)}{\partial A_{1n}} \\ \frac{\partial f(A)}{\partial A_{21}} & \frac{\partial f(A)}{\partial A_{22}} & \dots & \frac{\partial f(A)}{\partial A_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f(A)}{\partial A_{m1}} & \frac{\partial f(A)}{\partial A_{m2}} & \dots & \frac{\partial f(A)}{\partial A_{mn}} \end{bmatrix}$$

i.e., an $m \times n$ matrix with

$$(\nabla_A f(A))_{ij} = \frac{\partial f(A)}{\partial A_{ij}}.$$

Note that the size of $\nabla_A f(A)$ is always the same as the size of A . So if, in particular, A is just a vector $x \in \mathbb{R}^n$,

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}.$$

It is very important to remember that the gradient of a function is *only* defined if the function is real-valued, that is, if it returns a scalar value. We can not, for example, take the gradient of Ax , $A \in \mathbb{R}^{n \times n}$ with respect to x , since this quantity is vector-valued.

It follows directly from the equivalent properties of partial derivatives that:

- $\nabla_x(f(x) + g(x)) = \nabla_x f(x) + \nabla_x g(x)$.
- For $t \in \mathbb{R}$, $\nabla_x(t f(x)) = t \nabla_x f(x)$.

In principle, gradients are a natural extension of partial derivatives to functions of multiple variables. In practice, however, working with gradients can sometimes be tricky for notational reasons. For example, suppose that $A \in \mathbb{R}^{m \times n}$ is a matrix of fixed coefficients and suppose that $b \in \mathbb{R}^m$ is a vector of fixed coefficients. Let $f : \mathbb{R}^m \rightarrow \mathbb{R}$ be the function defined by $f(z) = z^T z$, such that $\nabla_z f(z) = 2z$. But now, consider the expression,

$$\nabla f(Ax).$$

How should this expression be interpreted? There are at least two possibilities:

1. In the first interpretation, recall that $\nabla_z f(z) = 2z$. Here, we interpret $\nabla f(Ax)$ as evaluating the gradient at the point Ax , hence,

$$\nabla f(Ax) = 2(Ax) = 2Ax \in \mathbb{R}^m.$$

2. In the second interpretation, we consider the quantity $f(Ax)$ as a function of the input variables x . More formally, let $g(x) = f(Ax)$. Then in this interpretation,

$$\nabla f(Ax) = \nabla_x g(x) \in \mathbb{R}^n.$$

Here, we can see that these two interpretations are indeed different. One interpretation yields an m -dimensional vector as a result, while the other interpretation yields an n -dimensional vector as a result! How can we resolve this?

Here, the key is to make explicit the variables which we are differentiating with respect to. In the first case, we are differentiating the function f with respect to its arguments z and then substituting the argument Ax . In the second case, we are differentiating the composite function $g(x) = f(Ax)$ with respect to x directly. We denote the first case as $\nabla_z f(Ax)$ and the second case as $\nabla_x f(Ax)$.⁴ Keeping the notation clear is extremely important (as you'll find out in your homework, in fact!).

4.2 The Hessian

Suppose that $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a function that takes a vector in $\mathbb{R}^n$ and returns a real number. Then the **Hessian** matrix with respect to x , written $\nabla_x^2 f(x)$ or simply as H is the $n \times n$ matrix of partial derivatives,

$$\nabla_x^2 f(x) \in \mathbb{R}^{n \times n} = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(x)}{\partial x_2 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_2^2} & \cdots & \frac{\partial^2 f(x)}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \frac{\partial^2 f(x)}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{bmatrix}.$$

In other words, $\nabla_x^2 f(x) \in \mathbb{R}^{n \times n}$, with

$$(\nabla_x^2 f(x))_{ij} = \frac{\partial^2 f(x)}{\partial x_i \partial x_j}.$$

Note that the Hessian is always symmetric, since

$$\frac{\partial^2 f(x)}{\partial x_i \partial x_j} = \frac{\partial^2 f(x)}{\partial x_j \partial x_i}.$$

Similar to the gradient, the Hessian is defined only when $f(x)$ is real-valued.

It is natural to think of the gradient as the analogue of the first derivative for functions of vectors, and the Hessian as the analogue of the second derivative (and the symbols we use also suggest this relation). This intuition is generally correct, but there are a few caveats to keep in mind.

⁴A drawback to this notation that we will have to live with is the fact that in the first case, $\nabla_z f(Ax)$ it appears that we are differentiating with respect to a variable that does not even appear in the expression being differentiated! For this reason, the first case is often written as $\nabla f(Ax)$, and the fact that we are differentiating with respect to the arguments of f is understood. However, the second case is *always* written as $\nabla_x f(Ax)$.

First, for real-valued functions of one variable $f : \mathbb{R} \rightarrow \mathbb{R}$, it is a basic definition that the second derivative is the derivative of the first derivative, i.e.,

$$\frac{\partial^2 f(x)}{\partial x^2} = \frac{\partial}{\partial x} \frac{\partial}{\partial x} f(x).$$

However, for functions of a vector, the gradient of the function is a vector, and we cannot take the gradient of a vector — i.e.,

$$\nabla_x \nabla_x f(x) = \nabla_x \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}$$

and this expression is not defined. Therefore, it is *not* the case that the Hessian is the gradient of the gradient. However, this is *almost* true, in the following sense: If we look at the i th entry of the gradient $(\nabla_x f(x))_i = \partial f(x)/\partial x_i$, and take the gradient with respect to x we get

$$\nabla_x \frac{\partial f(x)}{\partial x_i} = \begin{bmatrix} \frac{\partial^2 f(x)}{\partial x_i \partial x_1} \\ \frac{\partial^2 f(x)}{\partial x_i \partial x_2} \\ \vdots \\ \frac{\partial^2 f(x)}{\partial x_i \partial x_n} \end{bmatrix}$$

which is the i th column (or row) of the Hessian. Therefore,

$$\nabla_x^2 f(x) = \begin{bmatrix} \nabla_x(\nabla_x f(x))_1 & \nabla_x(\nabla_x f(x))_2 & \cdots & \nabla_x(\nabla_x f(x))_n \end{bmatrix}.$$

If we don't mind being a little bit sloppy we can say that (essentially) $\nabla_x^2 f(x) = \nabla_x(\nabla_x f(x))^T$, so long as we understand that this really means taking the gradient of each entry of $(\nabla_x f(x))^T$, not the gradient of the whole vector.

Finally, note that while we can take the gradient with respect to a matrix $A \in \mathbb{R}^n$, for the purposes of this class we will only consider taking the Hessian with respect to a vector $x \in \mathbb{R}^n$. This is simply a matter of convenience (and the fact that none of the calculations we do require us to find the Hessian with respect to a matrix), since the Hessian with respect to a matrix would have to represent all the partial derivatives $\partial^2 f(A)/(\partial A_{ij} \partial A_{k\ell})$, and it is rather cumbersome to represent this as a matrix.

4.3 Gradients and Hessians of Quadratic and Linear Functions

Now let's try to determine the gradient and Hessian matrices for a few simple functions. It should be noted that all the gradients given here are special cases of the gradients given in the CS229 lecture notes.

For $x \in \mathbb{R}^n$, let $f(x) = b^T x$ for some known vector $b \in \mathbb{R}^n$. Then

$$f(x) = \sum_{i=1}^n b_i x_i$$

so

$$\frac{\partial f(x)}{\partial x_k} = \frac{\partial}{\partial x_k} \sum_{i=1}^n b_i x_i = b_k.$$

From this we can easily see that $\nabla_x b^T x = b$. This should be compared to the analogous situation in single variable calculus, where $\partial/(\partial x) ax = a$.

Now consider the quadratic function $f(x) = x^T A x$ for $A \in \mathbb{S}^n$. Remember that

$$f(x) = \sum_{i=1}^n \sum_{j=1}^n A_{ij} x_i x_j.$$

To take the partial derivative, we'll consider the terms including x_k and x_k^2 factors separately:

$$\begin{aligned} \frac{\partial f(x)}{\partial x_k} &= \frac{\partial}{\partial x_k} \sum_{i=1}^n \sum_{j=1}^n A_{ij} x_i x_j \\ &= \frac{\partial}{\partial x_k} \left[\sum_{i \neq k} \sum_{j \neq k} A_{ij} x_i x_j + \sum_{i \neq k} A_{ik} x_i x_k + \sum_{j \neq k} A_{kj} x_k x_j + A_{kk} x_k^2 \right] \\ &= \sum_{i \neq k} A_{ik} x_i + \sum_{j \neq k} A_{kj} x_j + 2A_{kk} x_k \\ &= \sum_{i=1}^n A_{ik} x_i + \sum_{j=1}^n A_{kj} x_j = 2 \sum_{i=1}^n A_{ki} x_i, \end{aligned}$$

where the last equality follows since A is symmetric (which we can safely assume, since it is appearing in a quadratic form). Note that the k th entry of $\nabla_x f(x)$ is just the inner product of the k th row of A and x . Therefore, $\nabla_x x^T A x = 2Ax$. Again, this should remind you of the analogous fact in single-variable calculus, that $\partial/(\partial x) ax^2 = 2ax$.

Finally, let's look at the Hessian of the quadratic function $f(x) = x^T A x$ (it should be obvious that the Hessian of a linear function $b^T x$ is zero). In this case,

$$\frac{\partial^2 f(x)}{\partial x_k \partial x_\ell} = \frac{\partial}{\partial x_k} \left[\frac{\partial f(x)}{\partial x_\ell} \right] = \frac{\partial}{\partial x_k} \left[2 \sum_{i=1}^n A_{\ell i} x_i \right] = 2A_{\ell k} = 2A_{k\ell}.$$

Therefore, it should be clear that $\nabla_x^2 x^T A x = 2A$, which should be entirely expected (and again analogous to the single-variable fact that $\partial^2/(\partial x^2) ax^2 = 2a$).

To recap,

- $\nabla_x b^T x = b$
- $\nabla_x x^T A x = 2Ax$ (if A symmetric)
- $\nabla_x^2 x^T A x = 2A$ (if A symmetric)

4.4 Least Squares

Let's apply the equations we obtained in the last section to derive the least squares equations. Suppose we are given matrices $A \in \mathbb{R}^{m \times n}$ (for simplicity we assume A is full rank) and a vector $b \in \mathbb{R}^m$ such that $b \notin \mathcal{R}(A)$. In this situation we will not be able to find a vector $x \in \mathbb{R}^n$, such that $Ax = b$, so instead we want to find a vector x such that Ax is as close as possible to b , as measured by the square of the Euclidean norm $\|Ax - b\|_2^2$.

Using the fact that $\|x\|_2^2 = x^T x$, we have

$$\begin{aligned}\|Ax - b\|_2^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T A x - 2b^T A x + b^T b\end{aligned}$$

Taking the gradient with respect to x we have, and using the properties we derived in the previous section

$$\begin{aligned}\nabla_x (x^T A^T A x - 2b^T A x + b^T b) &= \nabla_x x^T A^T A x - \nabla_x 2b^T A x + \nabla_x b^T b \\ &= 2A^T A x - 2A^T b\end{aligned}$$

Setting this last expression equal to zero and solving for x gives the normal equations

$$x = (A^T A)^{-1} A^T b$$

which is the same as what we derived in class.

4.5 Gradients of the Determinant

Now let's consider a situation where we find the gradient of a function with respect to a matrix, namely for $A \in \mathbb{R}^{n \times n}$, we want to find $\nabla_A |A|$. Recall from our discussion of determinants that

$$|A| = \sum_{i=1}^n (-1)^{i+j} A_{ij} |A_{\setminus i, \setminus j}| \quad (\text{for any } j \in 1, \dots, n)$$

so

$$\frac{\partial}{\partial A_{k\ell}} |A| = \frac{\partial}{\partial A_{k\ell}} \sum_{i=1}^n (-1)^{i+j} A_{ij} |A_{\setminus i, \setminus j}| = (-1)^{k+\ell} |A_{\setminus k, \setminus \ell}| = (\text{adj}(A))_{\ell k}.$$

From this it immediately follows from the properties of the adjoint that

$$\nabla_A |A| = (\text{adj}(A))^T = |A| A^{-T}.$$

Now let's consider the function $f : \mathbb{S}_{++}^n \rightarrow \mathbb{R}$, $f(A) = \log |A|$. Note that we have to restrict the domain of f to be the positive definite matrices, since this ensures that $|A| > 0$, so that the log of $|A|$ is a real number. In this case we can use the chain rule (nothing fancy, just the ordinary chain rule from single-variable calculus) to see that

$$\frac{\partial \log |A|}{\partial A_{ij}} = \frac{\partial \log |A|}{\partial |A|} \frac{\partial |A|}{\partial A_{ij}} = \frac{1}{|A|} \frac{\partial |A|}{\partial A_{ij}}.$$

From this it should be obvious that

$$\nabla_A \log |A| = \frac{1}{|A|} \nabla_A |A| = A^{-1},$$

where we can drop the transpose in the last expression because A is symmetric. Note the similarity to the single-valued case, where $\partial/(\partial x) \log x = 1/x$.

4.6 Eigenvalues as Optimization

Finally, we use matrix calculus to solve an optimization problem in a way that leads directly to eigenvalue/eigenvector analysis. Consider the following, equality constrained optimization problem:

$$\max_{x \in \mathbb{R}^n} x^T A x \quad \text{subject to } \|x\|_2^2 = 1$$

for a symmetric matrix $A \in \mathbb{S}^n$. A standard way of solving optimization problems with equality constraints is by forming the **Lagrangian**, an objective function that includes the equality constraints.⁵ The Lagrangian in this case can be given by

$$\mathcal{L}(x, \lambda) = x^T A x - \lambda x^T x$$

where λ is called the Lagrange multiplier associated with the equality constraint. It can be established that for x^* to be an optimal point to the problem, the gradient of the Lagrangian has to be zero at x^* (this is not the only condition, but it is required). That is,

$$\nabla_x \mathcal{L}(x, \lambda) = \nabla_x (x^T A x - \lambda x^T x) = 2A^T x - 2\lambda x = 0.$$

Notice that this is just the linear equation $Ax = \lambda x$. This shows that the only points which can possibly maximize (or minimize) $x^T A x$ assuming $x^T x = 1$ are the eigenvectors of A .

⁵Don't worry if you haven't seen Lagrangians before, as we will cover them in greater detail later in CS229.

Review of Probability Theory

Arian Maleki and Tom Do
Stanford University

Probability theory is the study of uncertainty. Through this class, we will be relying on concepts from probability theory for deriving machine learning algorithms. These notes attempt to cover the basics of probability theory at a level appropriate for CS 229. The mathematical theory of probability is very sophisticated, and delves into a branch of analysis known as **measure theory**. In these notes, we provide a basic treatment of probability that does not address these finer details.

1 Elements of probability

In order to define a probability on a set we need a few basic elements,

- **Sample space** Ω : The set of all the outcomes of a random experiment. Here, each outcome $\omega \in \Omega$ can be thought of as a complete description of the state of the real world at the end of the experiment.
- **Set of events** (or **event space**) $\mathcal{F}$: A set whose elements $A \in \mathcal{F}$ (called **events**) are subsets of Ω (i.e., $A \subseteq \Omega$ is a collection of possible outcomes of an experiment).¹
- **Probability measure**: A function $P : \mathcal{F} \rightarrow \mathbb{R}$ that satisfies the following properties,
 - $P(A) \geq 0$, for all $A \in \mathcal{F}$
 - $P(\Omega) = 1$
 - If $A_1, A_2, \dots$ are disjoint events (i.e., $A_i \cap A_j = \emptyset$ whenever $i \neq j$), then

$$P(\cup_i A_i) = \sum_i P(A_i)$$

These three properties are called the **Axioms of Probability**.

Example: Consider the event of tossing a six-sided die. The sample space is $\Omega = \{1, 2, 3, 4, 5, 6\}$. We can define different event spaces on this sample space. For example, the simplest event space is the trivial event space $\mathcal{F} = \{\emptyset, \Omega\}$. Another event space is the set of all subsets of Ω . For the first event space, the unique probability measure satisfying the requirements above is given by $P(\emptyset) = 0, P(\Omega) = 1$. For the second event space, one valid probability measure is to assign the probability of each set in the event space to be $\frac{i}{6}$ where i is the number of elements of that set; for example, $P(\{1, 2, 3, 4\}) = \frac{4}{6}$ and $P(\{1, 2, 3\}) = \frac{3}{6}$.

Properties:

- If $A \subseteq B \implies P(A) \leq P(B)$.
- $P(A \cap B) \leq \min(P(A), P(B))$.
- (Union Bound) $P(A \cup B) \leq P(A) + P(B)$.
- $P(\Omega \setminus A) = 1 - P(A)$.
- (Law of Total Probability) If $A_1, \dots, A_k$ are a set of disjoint events such that $\cup_{i=1}^k A_i = \Omega$, then
$$\sum_{i=1}^k P(A_i) = 1.$$

¹ $\mathcal{F}$ should satisfy three properties: (1) $\emptyset \in \mathcal{F}$; (2) $A \in \mathcal{F} \implies \Omega \setminus A \in \mathcal{F}$; and (3) $A_1, A_2, \dots \in \mathcal{F} \implies \cup_i A_i \in \mathcal{F}$.

1.1 Conditional probability and independence

Let B be an event with non-zero probability. The conditional probability of any event A given B is defined as,

$$P(A|B) \triangleq \frac{P(A \cap B)}{P(B)}$$

In other words, $P(A|B)$ is the probability measure of the event A after observing the occurrence of event B . Two events are called independent if and only if $P(A \cap B) = P(A)P(B)$ (or equivalently, $P(A|B) = P(A)$). Therefore, independence is equivalent to saying that observing B does not have any effect on the probability of A .

2 Random variables

Consider an experiment in which we flip 10 coins, and we want to know the number of coins that come up heads. Here, the elements of the sample space Ω are 10-length sequences of heads and tails. For example, we might have $w_0 = \langle H, H, T, H, T, H, H, T, T, T \rangle \in \Omega$. However, in practice, we usually do not care about the probability of obtaining any particular sequence of heads and tails. Instead we usually care about real-valued functions of outcomes, such as the number of heads that appear among our 10 tosses, or the length of the longest run of tails. These functions, under some technical conditions, are known as **random variables**.

More formally, a random variable X is a function $X : \Omega \rightarrow \mathbb{R}$.² Typically, we will denote random variables using upper case letters $X(\omega)$ or more simply X (where the dependence on the random outcome ω is implied). We will denote the value that a random variable may take on using lower case letters x .

Example: In our experiment above, suppose that $X(\omega)$ is the number of heads which occur in the sequence of tosses ω . Given that only 10 coins are tossed, $X(\omega)$ can take only a finite number of values, so it is known as a **discrete random variable**. Here, the probability of the set associated with a random variable X taking on some specific value k is

$$P(X = k) := P(\{\omega : X(\omega) = k\}).$$

Example: Suppose that $X(\omega)$ is a random variable indicating the amount of time it takes for a radioactive particle to decay. In this case, $X(\omega)$ takes on a infinite number of possible values, so it is called a **continuous random variable**. We denote the probability that X takes on a value between two real constants a and b (where $a < b$) as

$$P(a \leq X \leq b) := P(\{\omega : a \leq X(\omega) \leq b\}).$$

2.1 Cumulative distribution functions

In order to specify the probability measures used when dealing with random variables, it is often convenient to specify alternative functions (CDFs, PDFs, and PMFs) from which the probability measure governing an experiment immediately follows. In this section and the next two sections, we describe each of these types of functions in turn.

A **cumulative distribution function (CDF)** is a function $F_X : \mathbb{R} \rightarrow [0, 1]$ which specifies a probability measure as,

$$F_X(x) \triangleq P(X \leq x). \quad (1)$$

By using this function one can calculate the probability of any event in $\mathcal{F}$.³ Figure ?? shows a sample CDF function.

Properties:

²Technically speaking, not every function is not acceptable as a random variable. From a measure-theoretic perspective, random variables must be Borel-measurable functions. Intuitively, this restriction ensures that given a random variable and its underlying outcome space, one can implicitly define the each of the events of the event space as being sets of outcomes $\omega \in \Omega$ for which $X(\omega)$ satisfies some property (e.g., the event $\{\omega : X(\omega) \geq 3\}$).

³This is a remarkable fact and is actually a theorem that is proved in more advanced courses.

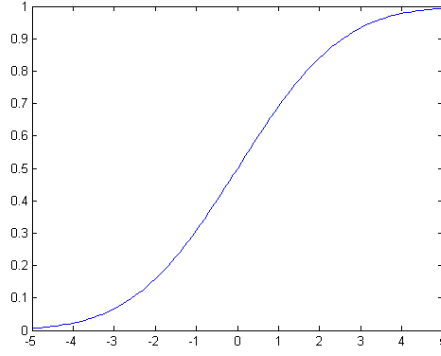


Figure 1: A cumulative distribution function (CDF).

- $0 \leq F_X(x) \leq 1$.
- $\lim_{x \rightarrow -\infty} F_X(x) = 0$.
- $\lim_{x \rightarrow \infty} F_X(x) = 1$.
- $x \leq y \implies F_X(x) \leq F_X(y)$.

2.2 Probability mass functions

When a random variable X takes on a finite set of possible values (i.e., X is a discrete random variable), a simpler way to represent the probability measure associated with a random variable is to directly specify the probability of each value that the random variable can assume. In particular, a *probability mass function (PMF)* is a function $p_X : \Omega \rightarrow \mathbb{R}$ such that

$$p_X(x) \triangleq P(X = x).$$

In the case of discrete random variable, we use the notation $Val(X)$ for the set of possible values that the random variable X may assume. For example, if $X(\omega)$ is a random variable indicating the number of heads out of ten tosses of coin, then $Val(X) = \{0, 1, 2, \dots, 10\}$.

Properties:

- $0 \leq p_X(x) \leq 1$.
- $\sum_{x \in Val(X)} p_X(x) = 1$.
- $\sum_{x \in A} p_X(x) = P(X \in A)$.

2.3 Probability density functions

For some continuous random variables, the cumulative distribution function $F_X(x)$ is differentiable everywhere. In these cases, we define the **Probability Density Function** or **PDF** as the derivative of the CDF, i.e.,

$$f_X(x) \triangleq \frac{dF_X(x)}{dx}. \quad (2)$$

Note here, that the PDF for a continuous random variable may not always exist (i.e., if $F_X(x)$ is not differentiable everywhere).

According to the properties of differentiation, for very small Δx ,

$$P(x \leq X \leq x + \Delta x) \approx f_X(x) \Delta x. \quad (3)$$

Both CDFs and PDFs (when they exist!) can be used for calculating the probabilities of different events. But it should be emphasized that the value of PDF at any given point x is not the probability

of that event, i.e., $f_X(x) \neq P(X = x)$. For example, $f_X(x)$ can take on values larger than one (but the integral of $f_X(x)$ over any subset of $\mathbb{R}$ will be at most one).

Properties:

- $f_X(x) \geq 0$.
- $\int_{-\infty}^{\infty} f_X(x) dx = 1$.
- $\int_{x \in A} f_X(x) dx = P(X \in A)$.

2.4 Expectation

Suppose that X is a discrete random variable with PMF $p_X(x)$ and $g : \mathbb{R} \rightarrow \mathbb{R}$ is an arbitrary function. In this case, $g(X)$ can be considered a random variable, and we define the **expectation** or **expected value** of $g(X)$ as

$$E[g(X)] \triangleq \sum_{x \in \text{Val}(X)} g(x)p_X(x).$$

If X is a continuous random variable with PDF $f_X(x)$, then the expected value of $g(X)$ is defined as,

$$E[g(X)] \triangleq \int_{-\infty}^{\infty} g(x)f_X(x)dx.$$

Intuitively, the expectation of $g(X)$ can be thought of as a “weighted average” of the values that $g(x)$ can take on for different values of x , where the weights are given by $p_X(x)$ or $f_X(x)$. As a special case of the above, note that the expectation, $E[X]$ of a random variable itself is found by letting $g(x) = x$; this is also known as the **mean** of the random variable X .

Properties:

- $E[a] = a$ for any constant $a \in \mathbb{R}$.
- $E[af(X)] = aE[f(X)]$ for any constant $a \in \mathbb{R}$.
- (Linearity of Expectation) $E[f(X) + g(X)] = E[f(X)] + E[g(X)]$.
- For a discrete random variable X , $E[1\{X = k\}] = P(X = k)$.

2.5 Variance

The **variance** of a random variable X is a measure of how concentrated the distribution of a random variable X is around its mean. Formally, the variance of a random variable X is defined as

$$\text{Var}[X] \triangleq E[(X - E(X))^2]$$

Using the properties in the previous section, we can derive an alternate expression for the variance:

$$\begin{aligned} E[(X - E[X])^2] &= E[X^2 - 2E[X]X + E[X]^2] \\ &= E[X^2] - 2E[X]E[X] + E[X]^2 \\ &= E[X^2] - E[X]^2, \end{aligned}$$

where the second equality follows from linearity of expectations and the fact that $E[X]$ is actually a constant with respect to the outer expectation.

Properties:

- $\text{Var}[a] = 0$ for any constant $a \in \mathbb{R}$.
- $\text{Var}[af(X)] = a^2\text{Var}[f(X)]$ for any constant $a \in \mathbb{R}$.

Example Calculate the mean and the variance of the uniform random variable X with PDF $f_X(x) = 1, \forall x \in [0, 1], 0$ elsewhere.

$$E[X] = \int_{-\infty}^{\infty} xf_X(x)dx = \int_0^1 xdx = \frac{1}{2}.$$

$$E[X^2] = \int_{-\infty}^{\infty} x^2 f_X(x) dx = \int_0^1 x^2 dx = \frac{1}{3}.$$

$$Var[X] = E[X^2] - E[X]^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12}.$$

Example: Suppose that $g(x) = 1\{x \in A\}$ for some subset $A \subseteq \Omega$. What is $E[g(X)]$?

Discrete case:

$$E[g(X)] = \sum_{x \in Val(X)} 1\{x \in A\} P_X(x) dx = \sum_{x \in A} P_X(x) dx = P(x \in A).$$

Continuous case:

$$E[g(X)] = \int_{-\infty}^{\infty} 1\{x \in A\} f_X(x) dx = \int_{x \in A} f_X(x) dx = P(x \in A).$$

2.6 Some common random variables

Discrete random variables

- $X \sim \text{Bernoulli}(p)$ (where $0 \leq p \leq 1$): one if a coin with heads probability p comes up heads, zero otherwise.

$$p(x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \end{cases}$$

- $X \sim \text{Binomial}(n, p)$ (where $0 \leq p \leq 1$): the number of heads in n independent flips of a coin with heads probability p .

$$p(x) = \binom{n}{x} p^x (1 - p)^{n-x}$$

- $X \sim \text{Geometric}(p)$ (where $p > 0$): the number of flips of a coin with heads probability p until the first heads.

$$p(x) = p(1 - p)^{x-1}$$

- $X \sim \text{Poisson}(\lambda)$ (where $\lambda > 0$): a probability distribution over the nonnegative integers used for modeling the frequency of rare events.

$$p(x) = e^{-\lambda} \frac{\lambda^x}{x!}$$

Continuous random variables

- $X \sim \text{Uniform}(a, b)$ (where $a < b$): equal probability density to every value between a and b on the real line.

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

- $X \sim \text{Exponential}(\lambda)$ (where $\lambda > 0$): decaying probability density over the nonnegative reals.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- $X \sim \text{Normal}(\mu, \sigma^2)$: also known as the Gaussian distribution

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

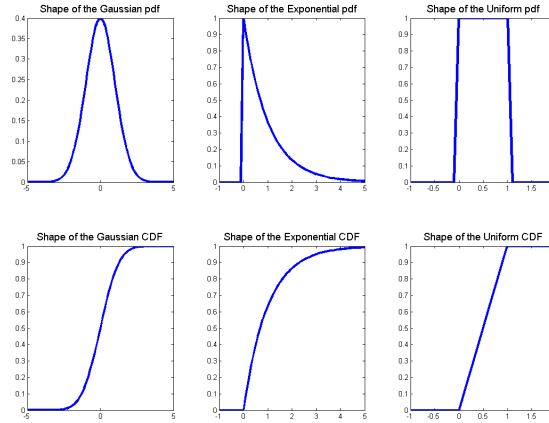


Figure 2: PDF and CDF of a couple of random variables.

The shape of the PDFs and CDFs of some of these random variables are shown in Figure ??.

The following table is the summary of some of the properties of these distributions.

Distribution	PDF or PMF	Mean	Variance
$Bernoulli(p)$	$\begin{cases} p, & \text{if } x = 1 \\ 1 - p, & \text{if } x = 0. \end{cases}$	p	$p(1 - p)$
$Binomial(n, p)$	$\binom{n}{k} p^k (1 - p)^{n-k}$ for $0 \leq k \leq n$	np	npq
$Geometric(p)$	$p(1 - p)^{k-1}$ for $k = 1, 2, \dots$	$\frac{1}{p}$	$\frac{1-p}{p^2}$
$Poisson(\lambda)$	$e^{-\lambda} \lambda^x / x!$ for $k = 1, 2, \dots$	λ	λ
$Uniform(a, b)$	$\frac{1}{b-a} \forall x \in (a, b)$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
$Gaussian(\mu, \sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	μ	σ^2
$Exponential(\lambda)$	$\lambda e^{-\lambda x}$ $x \geq 0, \lambda > 0$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$

3 Two random variables

Thus far, we have considered single random variables. In many situations, however, there may be more than one quantity that we are interested in knowing during a random experiment. For instance, in an experiment where we flip a coin ten times, we may care about both $X(\omega) =$ the number of heads that come up as well as $Y(\omega) =$ the length of the longest run of consecutive heads. In this section, we consider the setting of two random variables.

3.1 Joint and marginal distributions

Suppose that we have two random variables X and Y . One way to work with these two random variables is to consider each of them separately. If we do that we will only need $F_X(x)$ and $F_Y(y)$. But if we want to know about the values that X and Y assume simultaneously during outcomes of a random experiment, we require a more complicated structure known as the **joint cumulative distribution function** of X and Y , defined by

$$F_{XY}(x, y) = P(X \leq x, Y \leq y)$$

It can be shown that by knowing the joint cumulative distribution function, the probability of any event involving X and Y can be calculated.

The joint CDF $F_{XY}(x, y)$ and the joint distribution functions $F_X(x)$ and $F_Y(y)$ of each variable separately are related by

$$\begin{aligned} F_X(x) &= \lim_{y \rightarrow \infty} F_{XY}(x, y) dy \\ F_Y(y) &= \lim_{x \rightarrow \infty} F_{XY}(x, y) dx. \end{aligned}$$

Here, we call $F_X(x)$ and $F_Y(y)$ the **marginal cumulative distribution functions** of $F_{XY}(x, y)$.

Properties:

- $0 \leq F_{XY}(x, y) \leq 1$.
- $\lim_{x, y \rightarrow \infty} F_{XY}(x, y) = 1$.
- $\lim_{x, y \rightarrow -\infty} F_{XY}(x, y) = 0$.
- $F_X(x) = \lim_{y \rightarrow \infty} F_{XY}(x, y)$.

3.2 Joint and marginal probability mass functions

If X and Y are discrete random variables, then the **joint probability mass function** $p_{XY} : \mathbb{R} \times \mathbb{R} \rightarrow [0, 1]$ is defined by

$$p_{XY}(x, y) = P(X = x, Y = y).$$

Here, $0 \leq p_{XY}(x, y) \leq 1$ for all x, y , and $\sum_{x \in \text{Val}(X)} \sum_{y \in \text{Val}(Y)} p_{XY}(x, y) = 1$.

How does the joint PMF over two variables relate to the probability mass function for each variable separately? It turns out that

$$p_X(x) = \sum_y p_{XY}(x, y).$$

and similarly for $p_Y(y)$. In this case, we refer to $p_X(x)$ as the **marginal probability mass function** of X . In statistics, the process of forming the marginal distribution with respect to one variable by summing out the other variable is often known as “marginalization.”

3.3 Joint and marginal probability density functions

Let X and Y be two continuous random variables with joint distribution function F_{XY} . In the case that $F_{XY}(x, y)$ is everywhere differentiable in both x and y , then we can define the **joint probability density function**,

$$f_{XY}(x, y) = \frac{\partial^2 F_{XY}(x, y)}{\partial x \partial y}.$$

Like in the single-dimensional case, $f_{XY}(x, y) \neq P(X = x, Y = y)$, but rather

$$\iint_{x \in A} f_{XY}(x, y) dx dy = P((X, Y) \in A).$$

Note that the values of the probability density function $f_{XY}(x, y)$ are always nonnegative, but they may be greater than 1. Nonetheless, it must be the case that $\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f_{XY}(x, y) = 1$.

Analogous to the discrete case, we define

$$f_X(x) = \int_{-\infty}^{\infty} f_{XY}(x, y) dy,$$

as the **marginal probability density function** (or **marginal density**) of X , and similarly for $f_Y(y)$.

3.4 Conditional distributions

Conditional distributions seek to answer the question, what is the probability distribution over Y , when we know that X must take on a certain value x ? In the discrete case, the conditional probability mass function of X given Y is simply

$$p_{Y|X}(y|x) = \frac{p_{XY}(x, y)}{p_X(x)},$$

assuming that $p_X(x) \neq 0$.

In the continuous case, the situation is technically a little more complicated because the probability that a continuous random variable X takes on a specific value x is equal to zero⁴. Ignoring this technical point, we simply define, by analogy to the discrete case, the *conditional probability density* of Y given $X = x$ to be

$$f_{Y|X}(y|x) = \frac{f_{XY}(x, y)}{f_X(x)},$$

provided $f_X(x) \neq 0$.

3.5 Bayes's rule

A useful formula that often arises when trying to derive expression for the conditional probability of one variable given another, is **Bayes's rule**.

In the case of discrete random variables X and Y ,

$$P_{Y|X}(y|x) = \frac{P_{XY}(x, y)}{P_X(x)} = \frac{P_{X|Y}(x|y)P_Y(y)}{\sum_{y' \in \text{Val}(Y)} P_{X|Y}(x|y')P_Y(y')}.$$

If the random variables X and Y are continuous,

$$f_{Y|X}(y|x) = \frac{f_{XY}(x, y)}{f_X(x)} = \frac{f_{X|Y}(x|y)f_Y(y)}{\int_{-\infty}^{\infty} f_{X|Y}(x|y')f_Y(y')dy'}.$$

3.6 Independence

Two random variables X and Y are **independent** if $F_{XY}(x, y) = F_X(x)F_Y(y)$ for all values of x and y . Equivalently,

- For discrete random variables, $p_{XY}(x, y) = p_X(x)p_Y(y)$ for all $x \in \text{Val}(X)$, $y \in \text{Val}(Y)$.
- For discrete random variables, $p_{Y|X}(y|x) = p_Y(y)$ whenever $p_X(x) \neq 0$ for all $y \in \text{Val}(Y)$.
- For continuous random variables, $f_{XY}(x, y) = f_X(x)f_Y(y)$ for all $x, y \in \mathbb{R}$.
- For continuous random variables, $f_{Y|X}(y|x) = f_Y(y)$ whenever $f_X(x) \neq 0$ for all $y \in \mathbb{R}$.

⁴To get around this, a more reasonable way to calculate the conditional CDF is,

$$F_{Y|X}(y, x) = \lim_{\Delta x \rightarrow 0} P(Y \leq y | x \leq X \leq x + \Delta x).$$

It can be easily seen that if $F(x, y)$ is differentiable in both x, y then,

$$F_{Y|X}(y, x) = \int_{-\infty}^y \frac{f_{X,Y}(x, \alpha)}{f_X(x)} d\alpha$$

and therefore we define the conditional PDF of Y given $X = x$ in the following way,

$$f_{Y|X}(y|x) = \frac{f_{XY}(x, y)}{f_X(x)}$$

Informally, two random variables X and Y are **independent** if “knowing” the value of one variable will never have any effect on the conditional probability distribution of the other variable, that is, you know all the information about the pair (X, Y) by just knowing $f(x)$ and $f(y)$. The following lemma formalizes this observation:

Lemma 3.1. *If X and Y are independent then for any subsets $A, B \subseteq \mathbb{R}$, we have,*

$$P(X \in A, Y \in B) = P(X \in A)P(Y \in B)$$

By using the above lemma one can prove that if X is independent of Y then any function of X is independent of any function of Y .

3.7 Expectation and covariance

Suppose that we have two discrete random variables X, Y and $g : \mathbf{R}^2 \rightarrow \mathbf{R}$ is a function of these two random variables. Then the expected value of g is defined in the following way,

$$E[g(X, Y)] \triangleq \sum_{x \in \text{Val}(X)} \sum_{y \in \text{Val}(Y)} g(x, y) p_{XY}(x, y).$$

For continuous random variables X, Y , the analogous expression is

$$E[g(X, Y)] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} g(x, y) f_{XY}(x, y) dx dy.$$

We can use the concept of expectation to study the relationship of two random variables with each other. In particular, the **covariance** of two random variables X and Y is defined as

$$\text{Cov}[X, Y] \triangleq E[(X - E[X])(Y - E[Y])]$$

Using an argument similar to that for variance, we can rewrite this as,

$$\begin{aligned} \text{Cov}[X, Y] &= E[(X - E[X])(Y - E[Y])] \\ &= E[XY - XE[Y] - YE[X] + E[X]E[Y]] \\ &= E[XY] - E[X]E[Y] - E[Y]E[X] + E[X]E[Y] \\ &= E[XY] - E[X]E[Y]. \end{aligned}$$

Here, the key step in showing the equality of the two forms of covariance is in the third equality, where we use the fact that $E[X]$ and $E[Y]$ are actually constants which can be pulled out of the expectation. When $\text{Cov}[X, Y] = 0$, we say that X and Y are **uncorrelated**⁵.

Properties:

- (Linearity of expectation) $E[f(X, Y) + g(X, Y)] = E[f(X, Y)] + E[g(X, Y)]$.
- $\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] + 2\text{Cov}[X, Y]$.
- If X and Y are independent, then $\text{Cov}[X, Y] = 0$.
- If X and Y are independent, then $E[f(X)g(Y)] = E[f(X)]E[g(Y)]$.

4 Multiple random variables

The notions and ideas introduced in the previous section can be generalized to more than two random variables. In particular, suppose that we have n continuous random variables, $X_1(\omega), X_2(\omega), \dots, X_n(\omega)$. In this section, for simplicity of presentation, we focus only on the continuous case, but the generalization to discrete random variables works similarly.

⁵However, this is not the same thing as stating that X and Y are independent! For example, if $X \sim \text{Uniform}(-1, 1)$ and $Y = X^2$, then one can show that X and Y are uncorrelated, even though they are not independent.

4.1 Basic properties

We can define the **joint distribution function** of $X_1, X_2, \dots, X_n$, the **joint probability density function** of $X_1, X_2, \dots, X_n$, the **marginal probability density function** of X_1 , and the **conditional probability density function** of X_1 given $X_2, \dots, X_n$, as

$$\begin{aligned} F_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n) &= P(X_1 \leq x_1, X_2 \leq x_2, \dots, X_n \leq x_n) \\ f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n) &= \frac{\partial^n F_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n)}{\partial x_1 \dots \partial x_n} \\ f_{X_1}(X_1) &= \int_{-\infty}^{\infty} \dots \int_{-\infty}^{\infty} f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n) dx_2 \dots dx_n \\ f_{X_1|X_2, \dots, X_n}(x_1|x_2, \dots, x_n) &= \frac{f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n)}{f_{X_2, \dots, X_n}(x_2, \dots, x_n)} \end{aligned}$$

To calculate the probability of an event $A \subseteq \mathbb{R}^n$ we have,

$$P((x_1, x_2, \dots, x_n) \in A) = \int_{(x_1, x_2, \dots, x_n) \in A} f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n) dx_1 dx_2 \dots dx_n \quad (4)$$

Chain rule: From the definition of conditional probabilities for multiple random variables, one can show that

$$\begin{aligned} f(x_1, x_2, \dots, x_n) &= f(x_n|x_1, x_2, \dots, x_{n-1})f(x_1, x_2, \dots, x_{n-1}) \\ &= f(x_n|x_1, x_2, \dots, x_{n-1})f(x_{n-1}|x_1, x_2, \dots, x_{n-2})f(x_1, x_2, \dots, x_{n-2}) \\ &= \dots = f(x_1) \prod_{i=2}^n f(x_i|x_1, \dots, x_{i-1}). \end{aligned}$$

Independence: For multiple events, $A_1, \dots, A_k$, we say that $A_1, \dots, A_k$ are **mutually independent** if for any subset $S \subseteq \{1, 2, \dots, k\}$, we have

$$P(\cap_{i \in S} A_i) = \prod_{i \in S} P(A_i).$$

Likewise, we say that random variables $X_1, \dots, X_n$ are independent if

$$f(x_1, \dots, x_n) = f(x_1)f(x_2) \dots f(x_n).$$

Here, the definition of mutual independence is simply the natural generalization of independence of two random variables to multiple random variables.

Independent random variables arise often in machine learning algorithms where we assume that the training examples belonging to the training set represent independent samples from some unknown probability distribution. To make the significance of independence clear, consider a “bad” training set in which we first sample a single training example $(x^{(1)}, y^{(1)})$ from the some unknown distribution, and then add $m - 1$ copies of the exact same training example to the training set. In this case, we have (with some abuse of notation)

$$P((x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})) \neq \prod_{i=1}^m P(x^{(i)}, y^{(i)}).$$

Despite the fact that the training set has size m , the examples are not independent! While clearly the procedure described here is not a sensible method for building a training set for a machine learning algorithm, it turns out that in practice, non-independence of samples does come up often, and it has the effect of reducing the “effective size” of the training set.

4.2 Random vectors

Suppose that we have n random variables. When working with all these random variables together, we will often find it convenient to put them in a vector $X = [X_1 \ X_2 \ \dots \ X_n]^T$. We call the resulting vector a **random vector** (more formally, a random vector is a mapping from Ω to $\mathbb{R}^n$). It should be clear that random vectors are simply an alternative notation for dealing with n random variables, so the notions of joint PDF and CDF will apply to random vectors as well.

Expectation: Consider an arbitrary function from $g : \mathbb{R}^n \rightarrow \mathbb{R}$. The expected value of this function is defined as

$$E[g(X)] = \int_{\mathbb{R}^n} g(x_1, x_2, \dots, x_n) f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n) dx_1 dx_2 \dots dx_n, \quad (5)$$

where $\int_{\mathbb{R}^n}$ is n consecutive integrations from $-\infty$ to ∞ . If g is a function from $\mathbb{R}^n$ to $\mathbb{R}^m$, then the expected value of g is the element-wise expected values of the output vector, i.e., if g is

$$g(x) = \begin{bmatrix} g_1(x) \\ g_2(x) \\ \vdots \\ g_m(x) \end{bmatrix},$$

Then,

$$E[g(X)] = \begin{bmatrix} E[g_1(X)] \\ E[g_2(X)] \\ \vdots \\ E[g_m(X)] \end{bmatrix}.$$

Covariance matrix: For a given random vector $X : \Omega \rightarrow \mathbb{R}^n$, its covariance matrix Σ is the $n \times n$ square matrix whose entries are given by $\Sigma_{ij} = \text{Cov}[X_i, X_j]$.

From the definition of covariance, we have

$$\begin{aligned} \Sigma &= \begin{bmatrix} \text{Cov}[X_1, X_1] & \dots & \text{Cov}[X_1, X_n] \\ \vdots & \ddots & \vdots \\ \text{Cov}[X_n, X_1] & \dots & \text{Cov}[X_n, X_n] \end{bmatrix} \\ &= \begin{bmatrix} E[X_1^2] - E[X_1]E[X_1] & \dots & E[X_1 X_n] - E[X_1]E[X_n] \\ \vdots & \ddots & \vdots \\ E[X_n X_1] - E[X_n]E[X_1] & \dots & E[X_n^2] - E[X_n]E[X_n] \end{bmatrix} \\ &= \begin{bmatrix} E[X_1^2] & \dots & E[X_1 X_n] \\ \vdots & \ddots & \vdots \\ E[X_n X_1] & \dots & E[X_n^2] \end{bmatrix} - \begin{bmatrix} E[X_1]E[X_1] & \dots & E[X_1]E[X_n] \\ \vdots & \ddots & \vdots \\ E[X_n]E[X_1] & \dots & E[X_n]E[X_n] \end{bmatrix} \\ &= E[XX^T] - E[X]E[X]^T = \dots = E[(X - E[X])(X - E[X])^T]. \end{aligned}$$

where the matrix expectation is defined in the obvious way.

The covariance matrix has a number of useful properties:

- $\Sigma \succeq 0$; that is, Σ is positive semidefinite.
- $\Sigma = \Sigma^T$; that is, Σ is symmetric.

4.3 The multivariate Gaussian distribution

One particularly important example of a probability distribution over random vectors X is called the **multivariate Gaussian** or **multivariate normal** distribution. A random vector $X \in \mathbb{R}^n$ is said to have a multivariate normal (or Gaussian) distribution with mean $\mu \in \mathbb{R}^n$ and covariance matrix $\Sigma \in \mathbb{S}_{++}^n$ (where $\mathbb{S}_{++}^n$ refers to the space of symmetric positive definite $n \times n$ matrices)

$$f_{X_1, X_2, \dots, X_n}(x_1, x_2, \dots, x_n; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right).$$

We write this as $X \sim \mathcal{N}(\mu, \Sigma)$. Notice that in the case $n = 1$, this reduces the regular definition of a normal distribution with mean parameter μ_1 and variance Σ_{11} .

Generally speaking, Gaussian random variables are extremely useful in machine learning and statistics for two main reasons. First, they are extremely common when modeling “noise” in statistical algorithms. Quite often, noise can be considered to be the accumulation of a large number of small independent random perturbations affecting the measurement process; by the Central Limit Theorem, summations of independent random variables will tend to “look Gaussian.” Second, Gaussian random variables are convenient for many analytical manipulations, because many of the integrals involving Gaussian distributions that arise in practice have simple closed form solutions. We will encounter this later in the course.

5 Other resources

A good textbook on probability at the level needed for CS229 is the book, *A First Course on Probability* by Sheldon Ross.

CS229 Lecture notes

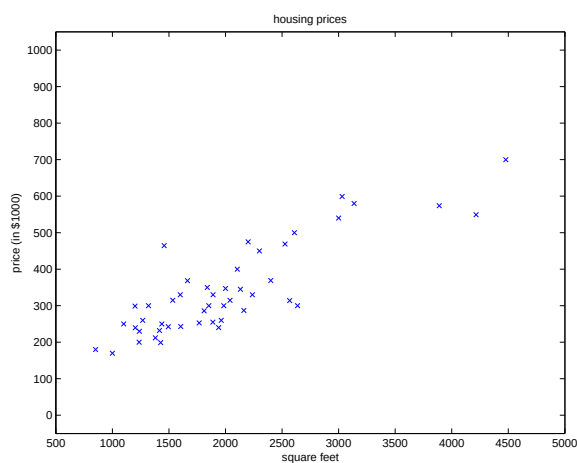
Andrew Ng

Supervised learning

Let's start by talking about a few examples of supervised learning problems. Suppose we have a dataset giving the living areas and prices of 47 houses from Portland, Oregon:

Living area (feet ²)	Price (1000\$)
2104	400
1600	330
2400	369
1416	232
3000	540
$\vdots$	$\vdots$

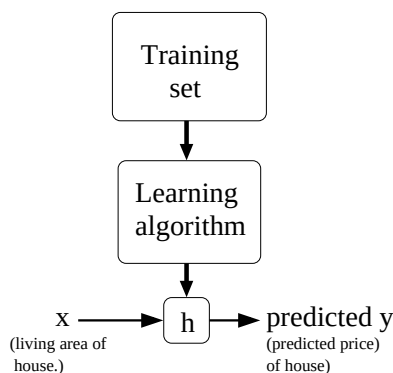
We can plot this data:



Given data like this, how can we learn to predict the prices of other houses in Portland, as a function of the size of their living areas?

To establish notation for future use, we'll use $x^{(i)}$ to denote the “input” variables (living area in this example), also called input **features**, and $y^{(i)}$ to denote the “output” or **target** variable that we are trying to predict (price). A pair $(x^{(i)}, y^{(i)})$ is called a **training example**, and the dataset that we'll be using to learn—a list of m training examples $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$ —is called a **training set**. Note that the superscript “ (i) ” in the notation is simply an index into the training set, and has nothing to do with exponentiation. We will also use $\mathcal{X}$ denote the space of input values, and $\mathcal{Y}$ the space of output values. In this example, $\mathcal{X} = \mathcal{Y} = \mathbb{R}$.

To describe the supervised learning problem slightly more formally, our goal is, given a training set, to learn a function $h : \mathcal{X} \mapsto \mathcal{Y}$ so that $h(x)$ is a “good” predictor for the corresponding value of y . For historical reasons, this function h is called a **hypothesis**. Seen pictorially, the process is therefore like this:



When the target variable that we're trying to predict is continuous, such as in our housing example, we call the learning problem a **regression** problem. When y can take on only a small number of discrete values (such as if, given the living area, we wanted to predict if a dwelling is a house or an apartment, say), we call it a **classification** problem.

Part I

Linear Regression

To make our housing example more interesting, let's consider a slightly richer dataset in which we also know the number of bedrooms in each house:

Living area (feet ²)	#bedrooms	Price (1000\$)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
$\vdots$	$\vdots$	$\vdots$

Here, the x 's are two-dimensional vectors in $\mathbb{R}^2$. For instance, $x_1^{(i)}$ is the living area of the i -th house in the training set, and $x_2^{(i)}$ is its number of bedrooms. (In general, when designing a learning problem, it will be up to you to decide what features to choose, so if you are out in Portland gathering housing data, you might also decide to include other features such as whether each house has a fireplace, the number of bathrooms, and so on. We'll say more about feature selection later, but for now let's take the features as given.)

To perform supervised learning, we must decide how we're going to represent functions/hypotheses h in a computer. As an initial choice, let's say we decide to approximate y as a linear function of x :

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Here, the θ_i 's are the **parameters** (also called **weights**) parameterizing the space of linear functions mapping from $\mathcal{X}$ to $\mathcal{Y}$. When there is no risk of confusion, we will drop the θ subscript in $h_{\theta}(x)$, and write it more simply as $h(x)$. To simplify our notation, we also introduce the convention of letting $x_0 = 1$ (this is the **intercept term**), so that

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x,$$

where on the right-hand side above we are viewing θ and x both as vectors, and here n is the number of input variables (not counting x_0).

Now, given a training set, how do we pick, or learn, the parameters θ ? One reasonable method seems to be to make $h(x)$ close to y , at least for the training examples we have. To formalize this, we will define a function that measures, for each value of the θ 's, how close the $h(x^{(i)})$'s are to the corresponding $y^{(i)}$'s. We define the **cost function**:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2.$$

If you've seen linear regression before, you may recognize this as the familiar least-squares cost function that gives rise to the **ordinary least squares** regression model. Whether or not you have seen it previously, let's keep going, and we'll eventually show this to be a special case of a much broader family of algorithms.

1 LMS algorithm

We want to choose θ so as to minimize $J(\theta)$. To do so, let's use a search algorithm that starts with some "initial guess" for θ , and that repeatedly changes θ to make $J(\theta)$ smaller, until hopefully we converge to a value of θ that minimizes $J(\theta)$. Specifically, let's consider the **gradient descent** algorithm, which starts with some initial θ , and repeatedly performs the update:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta).$$

(This update is simultaneously performed for all values of $j = 0, \dots, n$.) Here, α is called the **learning rate**. This is a very natural algorithm that repeatedly takes a step in the direction of steepest decrease of J .

In order to implement this algorithm, we have to work out what is the partial derivative term on the right hand side. Let's first work it out for the case of if we have only one training example (x, y) , so that we can neglect the sum in the definition of J . We have:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

For a single training example, this gives the update rule:¹

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}.$$

The rule is called the **LMS** update rule (LMS stands for “least mean squares”), and is also known as the **Widrow-Hoff** learning rule. This rule has several properties that seem natural and intuitive. For instance, the magnitude of the update is proportional to the **error** term $(y^{(i)} - h_\theta(x^{(i)}))$; thus, for instance, if we are encountering a training example on which our prediction nearly matches the actual value of $y^{(i)}$, then we find that there is little need to change the parameters; in contrast, a larger change to the parameters will be made if our prediction $h_\theta(x^{(i)})$ has a large error (i.e., if it is very far from $y^{(i)}$).

We’d derived the LMS rule for when there was only a single training example. There are two ways to modify this method for a training set of more than one example. The first is replace it with the following algorithm:

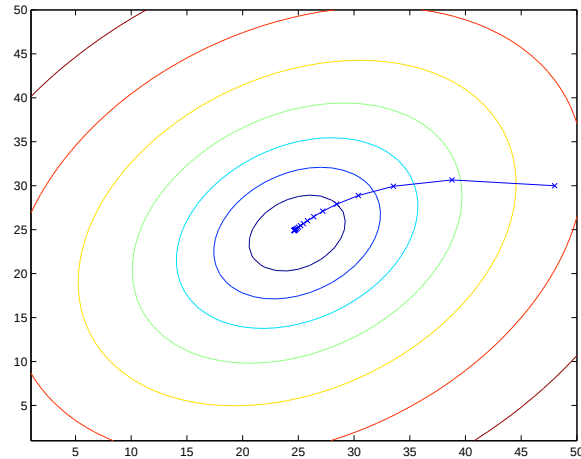
Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)} \quad (\text{for every } j).$$

}

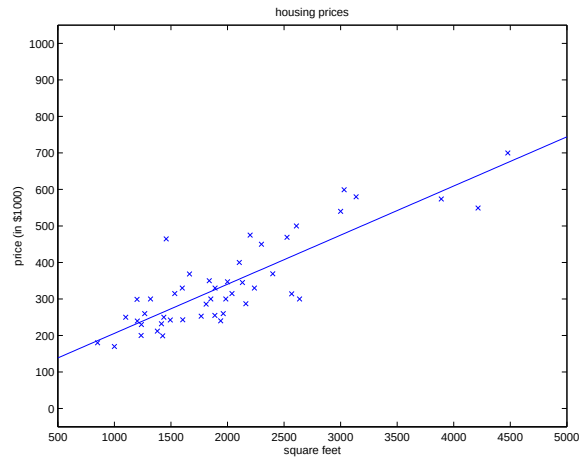
The reader can easily verify that the quantity in the summation in the update rule above is just $\partial J(\theta)/\partial \theta_j$ (for the original definition of J). So, this is simply gradient descent on the original cost function J . This method looks at every example in the entire training set on every step, and is called **batch gradient descent**. Note that, while gradient descent can be susceptible to local minima in general, the optimization problem we have posed here for linear regression has only one global, and no other local, optima; thus gradient descent always converges (assuming the learning rate α is not too large) to the global minimum. Indeed, J is a convex quadratic function. Here is an example of gradient descent as it is run to minimize a quadratic function.

¹We use the notation “ $a := b$ ” to denote an operation (in a computer program) in which we *set* the value of a variable a to be equal to the value of b . In other words, this operation overwrites a with the value of b . In contrast, we will write “ $a = b$ ” when we are asserting a statement of fact, that the value of a is equal to the value of b .



The ellipses shown above are the contours of a quadratic function. Also shown is the trajectory taken by gradient descent, which was initialized at $(48, 30)$. The x 's in the figure (joined by straight lines) mark the successive values of θ that gradient descent went through.

When we run batch gradient descent to fit θ on our previous dataset, to learn to predict housing price as a function of living area, we obtain $\theta_0 = 71.27$, $\theta_1 = 0.1345$. If we plot $h_\theta(x)$ as a function of x (area), along with the training data, we obtain the following figure:



If the number of bedrooms were included as one of the input features as well, we get $\theta_0 = 89.60$, $\theta_1 = 0.1392$, $\theta_2 = -8.738$.

The above results were obtained with batch gradient descent. There is an alternative to batch gradient descent that also works very well. Consider the following algorithm:

```

Loop {
    for i=1 to m, {
         $\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$     (for every  $j$ ).
    }
}

```

In this algorithm, we repeatedly run through the training set, and each time we encounter a training example, we update the parameters according to the gradient of the error with respect to that single training example only. This algorithm is called **stochastic gradient descent** (also **incremental gradient descent**). Whereas batch gradient descent has to scan through the entire training set before taking a single step—a costly operation if m is large—stochastic gradient descent can start making progress right away, and continues to make progress with each example it looks at. Often, stochastic gradient descent gets θ “close” to the minimum much faster than batch gradient descent. (Note however that it may never “converge” to the minimum, and the parameters θ will keep oscillating around the minimum of $J(\theta)$; but in practice most of the values near the minimum will be reasonably good approximations to the true minimum.²) For these reasons, particularly when the training set is large, stochastic gradient descent is often preferred over batch gradient descent.

2 The normal equations

Gradient descent gives one way of minimizing J . Let’s discuss a second way of doing so, this time performing the minimization explicitly and without resorting to an iterative algorithm. In this method, we will minimize J by explicitly taking its derivatives with respect to the θ_j ’s, and setting them to zero. To enable us to do this without having to write reams of algebra and pages full of matrices of derivatives, let’s introduce some notation for doing calculus with matrices.

²While it is more common to run stochastic gradient descent as we have described it and with a fixed learning rate α , by slowly letting the learning rate α decrease to zero as the algorithm runs, it is also possible to ensure that the parameters will converge to the global minimum rather than merely oscillate around the minimum.

2.1 Matrix derivatives

For a function $f : \mathbb{R}^{m \times n} \mapsto \mathbb{R}$ mapping from m -by- n matrices to the real numbers, we define the derivative of f with respect to A to be:

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f}{\partial A_{m1}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}$$

Thus, the gradient $\nabla_A f(A)$ is itself an m -by- n matrix, whose (i, j) -element is $\partial f / \partial A_{ij}$. For example, suppose $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ is a 2-by-2 matrix, and the function $f : \mathbb{R}^{2 \times 2} \mapsto \mathbb{R}$ is given by

$$f(A) = \frac{3}{2}A_{11} + 5A_{12}^2 + A_{21}A_{22}.$$

Here, A_{ij} denotes the (i, j) entry of the matrix A . We then have

$$\nabla_A f(A) = \begin{bmatrix} \frac{3}{2} & 10A_{12} \\ A_{22} & A_{21} \end{bmatrix}.$$

We also introduce the **trace** operator, written “tr.” For an n -by- n (square) matrix A , the trace of A is defined to be the sum of its diagonal entries:

$$\text{tr} A = \sum_{i=1}^n A_{ii}$$

If a is a real number (i.e., a 1-by-1 matrix), then $\text{tr} a = a$. (If you haven’t seen this “operator notation” before, you should think of the trace of A as $\text{tr}(A)$, or as application of the “trace” function to the matrix A . It’s more commonly written without the parentheses, however.)

The trace operator has the property that for two matrices A and B such that AB is square, we have that $\text{tr} AB = \text{tr} BA$. (Check this yourself!) As corollaries of this, we also have, e.g.,

$$\text{tr} ABC = \text{tr} CAB = \text{tr} BCA,$$

$$\text{tr} ABCD = \text{tr} DABC = \text{tr} CDAB = \text{tr} BCDA.$$

The following properties of the trace operator are also easily verified. Here, A and B are square matrices, and a is a real number:

$$\text{tr} A = \text{tr} A^T$$

$$\text{tr}(A + B) = \text{tr} A + \text{tr} B$$

$$\text{tr} aA = a \text{tr} A$$

We now state without proof some facts of matrix derivatives (we won't need some of these until later this quarter). Equation (4) applies only to non-singular square matrices A , where $|A|$ denotes the determinant of A . We have:

$$\nabla_A \text{tr} AB = B^T \quad (1)$$

$$\nabla_{A^T} f(A) = (\nabla_A f(A))^T \quad (2)$$

$$\nabla_A \text{tr} ABA^T C = CAB + C^T AB^T \quad (3)$$

$$\nabla_A |A| = |A|(A^{-1})^T. \quad (4)$$

To make our matrix notation more concrete, let us now explain in detail the meaning of the first of these equations. Suppose we have some fixed matrix $B \in \mathbb{R}^{n \times m}$. We can then define a function $f : \mathbb{R}^{m \times n} \mapsto \mathbb{R}$ according to $f(A) = \text{tr} AB$. Note that this definition makes sense, because if $A \in \mathbb{R}^{m \times n}$, then AB is a square matrix, and we can apply the trace operator to it; thus, f does indeed map from $\mathbb{R}^{m \times n}$ to $\mathbb{R}$. We can then apply our definition of matrix derivatives to find $\nabla_A f(A)$, which will itself be an m -by- n matrix. Equation (1) above states that the (i, j) entry of this matrix will be given by the (i, j) -entry of B^T , or equivalently, by B_{ji} .

The proofs of Equations (1-3) are reasonably simple, and are left as an exercise to the reader. Equations (4) can be derived using the adjoint representation of the inverse of a matrix.³

2.2 Least squares revisited

Armed with the tools of matrix derivatives, let us now proceed to find in closed-form the value of θ that minimizes $J(\theta)$. We begin by re-writing J in matrix-vectorial notation.

Given a training set, define the **design matrix** X to be the m -by- n matrix (actually m -by- $n + 1$, if we include the intercept term) that contains

³If we define A' to be the matrix whose (i, j) element is $(-1)^{i+j}$ times the determinant of the square matrix resulting from deleting row i and column j from A , then it can be proved that $A^{-1} = (A')^T / |A|$. (You can check that this is consistent with the standard way of finding A^{-1} when A is a 2-by-2 matrix. If you want to see a proof of this more general result, see an intermediate or advanced linear algebra text, such as Charles Curtis, 1991, *Linear Algebra*, Springer.) This shows that $A' = |A|(A^{-1})^T$. Also, the determinant of a matrix can be written $|A| = \sum_j A_{ij} A'_{ij}$. Since $(A')_{ij}$ does not depend on A_{ij} (as can be seen from its definition), this implies that $(\partial / \partial A_{ij})|A| = A'_{ij}$. Putting all this together shows the result.

the training examples' input values in its rows:

$$X = \begin{bmatrix} \text{---} (x^{(1)})^T \text{---} \\ \text{---} (x^{(2)})^T \text{---} \\ \vdots \\ \text{---} (x^{(m)})^T \text{---} \end{bmatrix}.$$

Also, let $\vec{y}$ be the m -dimensional vector containing all the target values from the training set:

$$\vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}.$$

Now, since $h_{\theta}(x^{(i)}) = (x^{(i)})^T \theta$, we can easily verify that

$$\begin{aligned} X\theta - \vec{y} &= \begin{bmatrix} (x^{(1)})^T \theta \\ \vdots \\ (x^{(m)})^T \theta \end{bmatrix} - \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix} \\ &= \begin{bmatrix} h_{\theta}(x^{(1)}) - y^{(1)} \\ \vdots \\ h_{\theta}(x^{(m)}) - y^{(m)} \end{bmatrix}. \end{aligned}$$

Thus, using the fact that for a vector z , we have that $z^T z = \sum_i z_i^2$:

$$\begin{aligned} \frac{1}{2}(X\theta - \vec{y})^T (X\theta - \vec{y}) &= \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= J(\theta) \end{aligned}$$

Finally, to minimize J , let's find its derivatives with respect to θ . Combining Equations (2) and (3), we find that

$$\nabla_{A^T} \text{tr} A B A^T C = B^T A^T C^T + B A^T C \quad (5)$$

Hence,

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= \nabla_{\theta} \frac{1}{2} (X\theta - \vec{y})^T (X\theta - \vec{y}) \\
&= \frac{1}{2} \nabla_{\theta} (\theta^T X^T X \theta - \theta^T X^T \vec{y} - \vec{y}^T X \theta + \vec{y}^T \vec{y}) \\
&= \frac{1}{2} \nabla_{\theta} \text{tr} (\theta^T X^T X \theta - \theta^T X^T \vec{y} - \vec{y}^T X \theta + \vec{y}^T \vec{y}) \\
&= \frac{1}{2} \nabla_{\theta} (\text{tr} \theta^T X^T X \theta - 2 \text{tr} \vec{y}^T X \theta) \\
&= \frac{1}{2} (X^T X \theta + X^T X \theta - 2 X^T \vec{y}) \\
&= X^T X \theta - X^T \vec{y}
\end{aligned}$$

In the third step, we used the fact that the trace of a real number is just the real number; the fourth step used the fact that $\text{tr} A = \text{tr} A^T$, and the fifth step used Equation (5) with $A^T = \theta$, $B = B^T = X^T X$, and $C = I$, and Equation (1). To minimize J , we set its derivatives to zero, and obtain the **normal equations**:

$$X^T X \theta = X^T \vec{y}$$

Thus, the value of θ that minimizes $J(\theta)$ is given in closed form by the equation

$$\theta = (X^T X)^{-1} X^T \vec{y}.$$

3 Probabilistic interpretation

When faced with a regression problem, why might linear regression, and specifically why might the least-squares cost function J , be a reasonable choice? In this section, we will give a set of probabilistic assumptions, under which least-squares regression is derived as a very natural algorithm.

Let us assume that the target variables and the inputs are related via the equation

$$y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)},$$

where $\epsilon^{(i)}$ is an error term that captures either unmodeled effects (such as if there are some features very pertinent to predicting housing price, but that we'd left out of the regression), or random noise. Let us further assume that the $\epsilon^{(i)}$ are distributed IID (independently and identically distributed) according to a Gaussian distribution (also called a Normal distribution) with

mean zero and some variance σ^2 . We can write this assumption as “ $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$.” I.e., the density of $\epsilon^{(i)}$ is given by

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right).$$

This implies that

$$p(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right).$$

The notation “ $p(y^{(i)} | x^{(i)}; \theta)$ ” indicates that this is the distribution of $y^{(i)}$ given $x^{(i)}$ and parameterized by θ . Note that we should not condition on θ (“ $p(y^{(i)} | x^{(i)}, \theta)$ ”), since θ is not a random variable. We can also write the distribution of $y^{(i)}$ as $y^{(i)} | x^{(i)}; \theta \sim \mathcal{N}(\theta^T x^{(i)}, \sigma^2)$.

Given X (the design matrix, which contains all the $x^{(i)}$ ’s) and θ , what is the distribution of the $y^{(i)}$ ’s? The probability of the data is given by $p(\vec{y} | X; \theta)$. This quantity is typically viewed a function of $\vec{y}$ (and perhaps X), for a fixed value of θ . When we wish to explicitly view this as a function of θ , we will instead call it the **likelihood** function:

$$L(\theta) = L(\theta; X, \vec{y}) = p(\vec{y} | X; \theta).$$

Note that by the independence assumption on the $\epsilon^{(i)}$ ’s (and hence also the $y^{(i)}$ ’s given the $x^{(i)}$ ’s), this can also be written

$$\begin{aligned} L(\theta) &= \prod_{i=1}^m p(y^{(i)} | x^{(i)}; \theta) \\ &= \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right). \end{aligned}$$

Now, given this probabilistic model relating the $y^{(i)}$ ’s and the $x^{(i)}$ ’s, what is a reasonable way of choosing our best guess of the parameters θ ? The principal of **maximum likelihood** says that we should choose θ so as to make the data as high probability as possible. I.e., we should choose θ to maximize $L(\theta)$.

Instead of maximizing $L(\theta)$, we can also maximize any strictly increasing function of $L(\theta)$. In particular, the derivations will be a bit simpler if we

instead maximize the **log likelihood** $\ell(\theta)$:

$$\begin{aligned}
 \ell(\theta) &= \log L(\theta) \\
 &= \log \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
 &= \sum_{i=1}^m \log \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
 &= m \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2} \cdot \frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2.
 \end{aligned}$$

Hence, maximizing $\ell(\theta)$ gives the same answer as minimizing

$$\frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2,$$

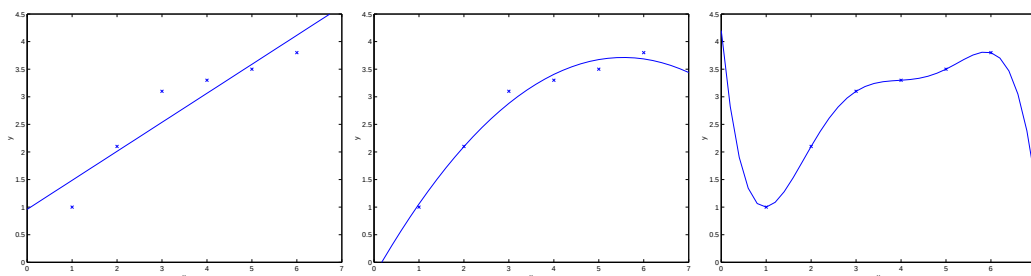
which we recognize to be $J(\theta)$, our original least-squares cost function.

To summarize: Under the previous probabilistic assumptions on the data, least-squares regression corresponds to finding the maximum likelihood estimate of θ . This is thus one set of assumptions under which least-squares regression can be justified as a very natural method that's just doing maximum likelihood estimation. (Note however that the probabilistic assumptions are by no means *necessary* for least-squares to be a perfectly good and rational procedure, and there may—and indeed there are—other natural assumptions that can also be used to justify it.)

Note also that, in our previous discussion, our final choice of θ did not depend on what was σ^2 , and indeed we'd have arrived at the same result even if σ^2 were unknown. We will use this fact again later, when we talk about the exponential family and generalized linear models.

4 Locally weighted linear regression

Consider the problem of predicting y from $x \in \mathbb{R}$. The leftmost figure below shows the result of fitting a $y = \theta_0 + \theta_1 x$ to a dataset. We see that the data doesn't really lie on straight line, and so the fit is not very good.



Instead, if we had added an extra feature x^2 , and fit $y = \theta_0 + \theta_1 x + \theta_2 x^2$, then we obtain a slightly better fit to the data. (See middle figure) Naively, it might seem that the more features we add, the better. However, there is also a danger in adding too many features: The rightmost figure is the result of fitting a 5-th order polynomial $y = \sum_{j=0}^5 \theta_j x^j$. We see that even though the fitted curve passes through the data perfectly, we would not expect this to be a very good predictor of, say, housing prices (y) for different living areas (x). Without formally defining what these terms mean, we'll say the figure on the left shows an instance of **underfitting**—in which the data clearly shows structure not captured by the model—and the figure on the right is an example of **overfitting**. (Later in this class, when we talk about learning theory we'll formalize some of these notions, and also define more carefully just what it means for a hypothesis to be good or bad.)

As discussed previously, and as shown in the example above, the choice of features is important to ensuring good performance of a learning algorithm. (When we talk about model selection, we'll also see algorithms for automatically choosing a good set of features.) In this section, let us briefly talk about the locally weighted linear regression (LWR) algorithm which, assuming there is sufficient training data, makes the choice of features less critical. This treatment will be brief, since you'll get a chance to explore some of the properties of the LWR algorithm yourself in the homework.

In the original linear regression algorithm, to make a prediction at a query point x (i.e., to evaluate $h(x)$), we would:

1. Fit θ to minimize $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$.
2. Output $\theta^T x$.

In contrast, the locally weighted linear regression algorithm does the following:

1. Fit θ to minimize $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$.
2. Output $\theta^T x$.

Here, the $w^{(i)}$'s are non-negative valued **weights**. Intuitively, if $w^{(i)}$ is large for a particular value of i , then in picking θ , we'll try hard to make $(y^{(i)} - \theta^T x^{(i)})^2$ small. If $w^{(i)}$ is small, then the $(y^{(i)} - \theta^T x^{(i)})^2$ error term will be pretty much ignored in the fit.

A fairly standard choice for the weights is⁴

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

Note that the weights depend on the particular point x at which we're trying to evaluate x . Moreover, if $|x^{(i)} - x|$ is small, then $w^{(i)}$ is close to 1; and if $|x^{(i)} - x|$ is large, then $w^{(i)}$ is small. Hence, θ is chosen giving a much higher “weight” to the (errors on) training examples close to the query point x . (Note also that while the formula for the weights takes a form that is cosmetically similar to the density of a Gaussian distribution, the $w^{(i)}$'s do not directly have anything to do with Gaussians, and in particular the $w^{(i)}$ are not random variables, normally distributed or otherwise.) The parameter τ controls how quickly the weight of a training example falls off with distance of its $x^{(i)}$ from the query point x ; τ is called the **bandwidth** parameter, and is also something that you'll get to experiment with in your homework.

Locally weighted linear regression is the first example we're seeing of a **non-parametric** algorithm. The (unweighted) linear regression algorithm that we saw earlier is known as a **parametric** learning algorithm, because it has a fixed, finite number of parameters (the θ_i 's), which are fit to the data. Once we've fit the θ_i 's and stored them away, we no longer need to keep the training data around to make future predictions. In contrast, to make predictions using locally weighted linear regression, we need to keep the entire training set around. The term “non-parametric” (roughly) refers to the fact that the amount of stuff we need to keep in order to represent the hypothesis h grows linearly with the size of the training set.

⁴If x is vector-valued, this is generalized to be $w^{(i)} = \exp(-(x^{(i)} - x)^T(x^{(i)} - x)/(2\tau^2))$, or $w^{(i)} = \exp(-(x^{(i)} - x)^T \Sigma^{-1}(x^{(i)} - x)/2)$, for an appropriate choice of τ or Σ .

Part II

Classification and logistic regression

Let's now talk about the classification problem. This is just like the regression problem, except that the values y we now want to predict take on only a small number of discrete values. For now, we will focus on the **binary classification** problem in which y can take on only two values, 0 and 1. (Most of what we say here will also generalize to the multiple-class case.) For instance, if we are trying to build a spam classifier for email, then $x^{(i)}$ may be some features of a piece of email, and y may be 1 if it is a piece of spam mail, and 0 otherwise. 0 is also called the **negative class**, and 1 the **positive class**, and they are sometimes also denoted by the symbols “-” and “+.” Given $x^{(i)}$, the corresponding $y^{(i)}$ is also called the **label** for the training example.

5 Logistic regression

We could approach the classification problem ignoring the fact that y is discrete-valued, and use our old linear regression algorithm to try to predict y given x . However, it is easy to construct examples where this method performs very poorly. Intuitively, it also doesn't make sense for $h_\theta(x)$ to take values larger than 1 or smaller than 0 when we know that $y \in \{0, 1\}$.

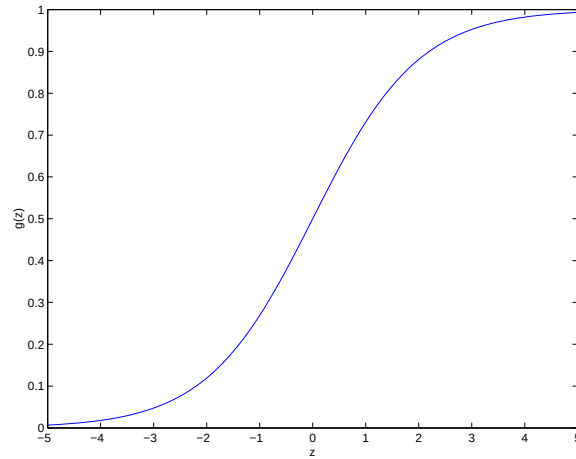
To fix this, let's change the form for our hypotheses $h_\theta(x)$. We will choose

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

where

$$g(z) = \frac{1}{1 + e^{-z}}$$

is called the **logistic function** or the **sigmoid function**. Here is a plot showing $g(z)$:



Notice that $g(z)$ tends towards 1 as $z \rightarrow \infty$, and $g(z)$ tends towards 0 as $z \rightarrow -\infty$. Moreover, $g(z)$, and hence also $h(x)$, is always bounded between 0 and 1. As before, we are keeping the convention of letting $x_0 = 1$, so that $\theta^T x = \theta_0 + \sum_{j=1}^n \theta_j x_j$.

For now, let's take the choice of g as given. Other functions that smoothly increase from 0 to 1 can also be used, but for a couple of reasons that we'll see later (when we talk about GLMs, and when we talk about generative learning algorithms), the choice of the logistic function is a fairly natural one. Before moving on, here's a useful property of the derivative of the sigmoid function, which we write as g' :

$$\begin{aligned}
 g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\
 &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\
 &= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{(1 + e^{-z})} \right) \\
 &= g(z)(1 - g(z)).
 \end{aligned}$$

So, given the logistic regression model, how do we fit θ for it? Following how we saw least squares regression could be derived as the maximum likelihood estimator under a set of assumptions, let's endow our classification model with a set of probabilistic assumptions, and then fit the parameters via maximum likelihood.

Let us assume that

$$\begin{aligned} P(y = 1 \mid x; \theta) &= h_\theta(x) \\ P(y = 0 \mid x; \theta) &= 1 - h_\theta(x) \end{aligned}$$

Note that this can be written more compactly as

$$p(y \mid x; \theta) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}$$

Assuming that the m training examples were generated independently, we can then write down the likelihood of the parameters as

$$\begin{aligned} L(\theta) &= p(\vec{y} \mid X; \theta) \\ &= \prod_{i=1}^m p(y^{(i)} \mid x^{(i)}; \theta) \\ &= \prod_{i=1}^m (h_\theta(x^{(i)}))^{y^{(i)}} (1 - h_\theta(x^{(i)}))^{1-y^{(i)}} \end{aligned}$$

As before, it will be easier to maximize the log likelihood:

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^m y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \end{aligned}$$

How do we maximize the likelihood? Similar to our derivation in the case of linear regression, we can use gradient ascent. Written in vectorial notation, our updates will therefore be given by $\theta := \theta + \alpha \nabla_\theta \ell(\theta)$. (Note the positive rather than negative sign in the update formula, since we're maximizing, rather than minimizing, a function now.) Let's start by working with just one training example (x, y) , and take derivatives to derive the stochastic gradient ascent rule:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} \ell(\theta) &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\ &= \left(y \frac{1}{g(\theta^T x)} - (1 - y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x) (1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\ &= (y(1 - g(\theta^T x)) - (1 - y)g(\theta^T x)) x_j \\ &= (y - h_\theta(x)) x_j \end{aligned}$$

Above, we used the fact that $g'(z) = g(z)(1 - g(z))$. This therefore gives us the stochastic gradient ascent rule

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}$$

If we compare this to the LMS update rule, we see that it looks identical; but this is *not* the same algorithm, because $h_\theta(x^{(i)})$ is now defined as a non-linear function of $\theta^T x^{(i)}$. Nonetheless, it's a little surprising that we end up with the same update rule for a rather different algorithm and learning problem. Is this coincidence, or is there a deeper reason behind this? We'll answer this when we get to GLM models. (See also the extra credit problem on Q3 of problem set 1.)

6 Digression: The perceptron learning algorithm

We now digress to talk briefly about an algorithm that's of some historical interest, and that we will also return to later when we talk about learning theory. Consider modifying the logistic regression method to “force” it to output values that are either 0 or 1 or exactly. To do so, it seems natural to change the definition of g to be the threshold function:

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

If we then let $h_\theta(x) = g(\theta^T x)$ as before but using this modified definition of g , and if we use the update rule

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_\theta(x^{(i)})) x_j^{(i)}.$$

then we have the **perceptron learning algorithm**.

In the 1960s, this “perceptron” was argued to be a rough model for how individual neurons in the brain work. Given how simple the algorithm is, it will also provide a starting point for our analysis when we talk about learning theory later in this class. Note however that even though the perceptron may be cosmetically similar to the other algorithms we talked about, it is actually a very different type of algorithm than logistic regression and least squares linear regression; in particular, it is difficult to endow the perceptron's predictions with meaningful probabilistic interpretations, or derive the perceptron as a maximum likelihood estimation algorithm.

7 Another algorithm for maximizing $\ell(\theta)$

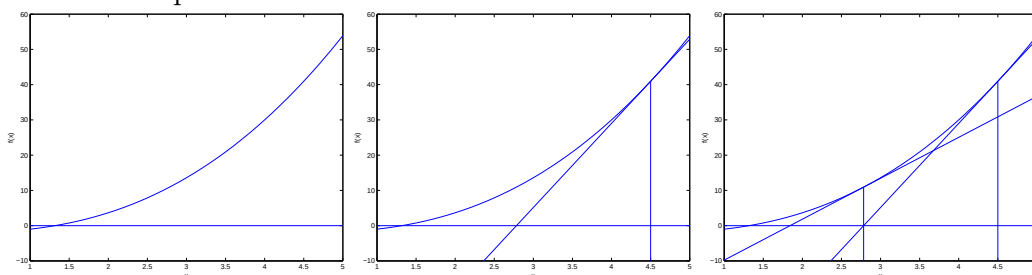
Returning to logistic regression with $g(z)$ being the sigmoid function, let's now talk about a different algorithm for maximizing $\ell(\theta)$.

To get us started, let's consider Newton's method for finding a zero of a function. Specifically, suppose we have some function $f : \mathbb{R} \mapsto \mathbb{R}$, and we wish to find a value of θ so that $f(\theta) = 0$. Here, $\theta \in \mathbb{R}$ is a real number. Newton's method performs the following update:

$$\theta := \theta - \frac{f(\theta)}{f'(\theta)}.$$

This method has a natural interpretation in which we can think of it as approximating the function f via a linear function that is tangent to f at the current guess θ , solving for where that linear function equals to zero, and letting the next guess for θ be where that linear function is zero.

Here's a picture of the Newton's method in action:



In the leftmost figure, we see the function f plotted along with the line $y = 0$. We're trying to find θ so that $f(\theta) = 0$; the value of θ that achieves this is about 1.3. Suppose we initialized the algorithm with $\theta = 4.5$. Newton's method then fits a straight line tangent to f at $\theta = 4.5$, and solves for the where that line evaluates to 0. (Middle figure.) This give us the next guess for θ , which is about 2.8. The rightmost figure shows the result of running one more iteration, which the updates θ to about 1.8. After a few more iterations, we rapidly approach $\theta = 1.3$.

Newton's method gives a way of getting to $f(\theta) = 0$. What if we want to use it to maximize some function ℓ ? The maxima of ℓ correspond to points where its first derivative $\ell'(\theta)$ is zero. So, by letting $f(\theta) = \ell'(\theta)$, we can use the same algorithm to maximize ℓ , and we obtain update rule:

$$\theta := \theta - \frac{\ell'(\theta)}{\ell''(\theta)}.$$

(Something to think about: How would this change if we wanted to use Newton's method to minimize rather than maximize a function?)

Lastly, in our logistic regression setting, θ is vector-valued, so we need to generalize Newton's method to this setting. The generalization of Newton's method to this multidimensional setting (also called the Newton-Raphson method) is given by

$$\theta := \theta - H^{-1} \nabla_{\theta} \ell(\theta).$$

Here, $\nabla_{\theta} \ell(\theta)$ is, as usual, the vector of partial derivatives of $\ell(\theta)$ with respect to the θ_i 's; and H is an n -by- n matrix (actually, $n + 1$ -by- $n + 1$, assuming that we include the intercept term) called the **Hessian**, whose entries are given by

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}.$$

Newton's method typically enjoys faster convergence than (batch) gradient descent, and requires many fewer iterations to get very close to the minimum. One iteration of Newton's can, however, be more expensive than one iteration of gradient descent, since it requires finding and inverting an n -by- n Hessian; but so long as n is not too large, it is usually much faster overall. When Newton's method is applied to maximize the logistic regression log likelihood function $\ell(\theta)$, the resulting method is also called **Fisher scoring**.

Part III

Generalized Linear Models⁵

So far, we've seen a regression example, and a classification example. In the regression example, we had $y|x; \theta \sim \mathcal{N}(\mu, \sigma^2)$, and in the classification one, $y|x; \theta \sim \text{Bernoulli}(\phi)$, for some appropriate definitions of μ and ϕ as functions of x and θ . In this section, we will show that both of these methods are special cases of a broader family of models, called Generalized Linear Models (GLMs). We will also show how other models in the GLM family can be derived and applied to other classification and regression problems.

8 The exponential family

To work our way up to GLMs, we will begin by defining exponential family distributions. We say that a class of distributions is in the exponential family if it can be written in the form

$$p(y; \eta) = b(y) \exp(\eta^T T(y) - a(\eta)) \quad (6)$$

Here, η is called the **natural parameter** (also called the **canonical parameter**) of the distribution; $T(y)$ is the **sufficient statistic** (for the distributions we consider, it will often be the case that $T(y) = y$); and $a(\eta)$ is the **log partition function**. The quantity $e^{-a(\eta)}$ essentially plays the role of a normalization constant, that makes sure the distribution $p(y; \eta)$ sums/integrates over y to 1.

A fixed choice of T , a and b defines a *family* (or set) of distributions that is parameterized by η ; as we vary η , we then get different distributions within this family.

We now show that the Bernoulli and the Gaussian distributions are examples of exponential family distributions. The Bernoulli distribution with mean ϕ , written $\text{Bernoulli}(\phi)$, specifies a distribution over $y \in \{0, 1\}$, so that $p(y = 1; \phi) = \phi$; $p(y = 0; \phi) = 1 - \phi$. As we vary ϕ , we obtain Bernoulli distributions with different means. We now show that this class of Bernoulli distributions, ones obtained by varying ϕ , is in the exponential family; i.e., that there is a choice of T , a and b so that Equation (6) becomes exactly the class of Bernoulli distributions.

⁵The presentation of the material in this section takes inspiration from Michael I. Jordan, *Learning in graphical models* (unpublished book draft), and also McCullagh and Nelder, *Generalized Linear Models* (2nd ed.).

We write the Bernoulli distribution as:

$$\begin{aligned}
 p(y; \phi) &= \phi^y (1 - \phi)^{1-y} \\
 &= \exp(y \log \phi + (1 - y) \log(1 - \phi)) \\
 &= \exp \left(\left(\log \left(\frac{\phi}{1 - \phi} \right) \right) y + \log(1 - \phi) \right).
 \end{aligned}$$

Thus, the natural parameter is given by $\eta = \log(\phi/(1 - \phi))$. Interestingly, if we invert this definition for η by solving for ϕ in terms of η , we obtain $\phi = 1/(1 + e^{-\eta})$. This is the familiar sigmoid function! This will come up again when we derive logistic regression as a GLM. To complete the formulation of the Bernoulli distribution as an exponential family distribution, we also have

$$\begin{aligned}
 T(y) &= y \\
 a(\eta) &= -\log(1 - \phi) \\
 &= \log(1 + e^\eta) \\
 b(y) &= 1
 \end{aligned}$$

This shows that the Bernoulli distribution can be written in the form of Equation (6), using an appropriate choice of T , a and b .

Let's now move on to consider the Gaussian distribution. Recall that, when deriving linear regression, the value of σ^2 had no effect on our final choice of θ and $h_\theta(x)$. Thus, we can choose an arbitrary value for σ^2 without changing anything. To simplify the derivation below, let's set $\sigma^2 = 1$.⁶ We then have:

$$\begin{aligned}
 p(y; \mu) &= \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2} (y - \mu)^2 \right) \\
 &= \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{1}{2} y^2 \right) \cdot \exp \left(\mu y - \frac{1}{2} \mu^2 \right)
 \end{aligned}$$

⁶If we leave σ^2 as a variable, the Gaussian distribution can also be shown to be in the exponential family, where $\eta \in \mathbb{R}^2$ is now a 2-dimension vector that depends on both μ and σ . For the purposes of GLMs, however, the σ^2 parameter can also be treated by considering a more general definition of the exponential family: $p(y; \eta, \tau) = b(a, \tau) \exp((\eta^T T(y) - a(\eta))/c(\tau))$. Here, τ is called the **dispersion parameter**, and for the Gaussian, $c(\tau) = \sigma^2$; but given our simplification above, we won't need the more general definition for the examples we will consider here.

Thus, we see that the Gaussian is in the exponential family, with

$$\begin{aligned}\eta &= \mu \\ T(y) &= y \\ a(\eta) &= \mu^2/2 \\ &= \eta^2/2 \\ b(y) &= (1/\sqrt{2\pi}) \exp(-y^2/2).\end{aligned}$$

There're many other distributions that are members of the exponential family: The multinomial (which we'll see later), the Poisson (for modelling count-data; also see the problem set); the gamma and the exponential (for modelling continuous, non-negative random variables, such as time-intervals); the beta and the Dirichlet (for distributions over probabilities); and many more. In the next section, we will describe a general “recipe” for constructing models in which y (given x and θ) comes from any of these distributions.

9 Constructing GLMs

Suppose you would like to build a model to estimate the number y of customers arriving in your store (or number of page-views on your website) in any given hour, based on certain features x such as store promotions, recent advertising, weather, day-of-week, etc. We know that the Poisson distribution usually gives a good model for numbers of visitors. Knowing this, how can we come up with a model for our problem? Fortunately, the Poisson is an exponential family distribution, so we can apply a Generalized Linear Model (GLM). In this section, we will describe a method for constructing GLM models for problems such as these.

More generally, consider a classification or regression problem where we would like to predict the value of some random variable y as a function of x . To derive a GLM for this problem, we will make the following three assumptions about the conditional distribution of y given x and about our model:

1. $y \mid x; \theta \sim \text{ExponentialFamily}(\eta)$. I.e., given x and θ , the distribution of y follows some exponential family distribution, with parameter η .
2. Given x , our goal is to predict the expected value of $T(y)$ given x . In most of our examples, we will have $T(y) = y$, so this means we would like the prediction $h(x)$ output by our learned hypothesis h to

satisfy $h(x) = E[y|x]$. (Note that this assumption is satisfied in the choices for $h_\theta(x)$ for both logistic regression and linear regression. For instance, in logistic regression, we had $h_\theta(x) = p(y = 1|x; \theta) = 0 \cdot p(y = 0|x; \theta) + 1 \cdot p(y = 1|x; \theta) = E[y|x; \theta]$.)

3. The natural parameter η and the inputs x are related linearly: $\eta = \theta^T x$. (Or, if η is vector-valued, then $\eta_i = \theta_i^T x$.)

The third of these assumptions might seem the least well justified of the above, and it might be better thought of as a “design choice” in our recipe for designing GLMs, rather than as an assumption per se. These three assumptions/design choices will allow us to derive a very elegant class of learning algorithms, namely GLMs, that have many desirable properties such as ease of learning. Furthermore, the resulting models are often very effective for modelling different types of distributions over y ; for example, we will shortly show that both logistic regression and ordinary least squares can both be derived as GLMs.

9.1 Ordinary Least Squares

To show that ordinary least squares is a special case of the GLM family of models, consider the setting where the target variable y (also called the **response variable** in GLM terminology) is continuous, and we model the conditional distribution of y given x as a Gaussian $\mathcal{N}(\mu, \sigma^2)$. (Here, μ may depend x .) So, we let the *ExponentialFamily*(η) distribution above be the Gaussian distribution. As we saw previously, in the formulation of the Gaussian as an exponential family distribution, we had $\mu = \eta$. So, we have

$$\begin{aligned} h_\theta(x) &= E[y|x; \theta] \\ &= \mu \\ &= \eta \\ &= \theta^T x. \end{aligned}$$

The first equality follows from Assumption 2, above; the second equality follows from the fact that $y|x; \theta \sim \mathcal{N}(\mu, \sigma^2)$, and so its expected value is given by μ ; the third equality follows from Assumption 1 (and our earlier derivation showing that $\mu = \eta$ in the formulation of the Gaussian as an exponential family distribution); and the last equality follows from Assumption 3.

9.2 Logistic Regression

We now consider logistic regression. Here we are interested in binary classification, so $y \in \{0, 1\}$. Given that y is binary-valued, it therefore seems natural to choose the Bernoulli family of distributions to model the conditional distribution of y given x . In our formulation of the Bernoulli distribution as an exponential family distribution, we had $\phi = 1/(1 + e^{-\eta})$. Furthermore, note that if $y|x; \theta \sim \text{Bernoulli}(\phi)$, then $E[y|x; \theta] = \phi$. So, following a similar derivation as the one for ordinary least squares, we get:

$$\begin{aligned} h_{\theta}(x) &= E[y|x; \theta] \\ &= \phi \\ &= 1/(1 + e^{-\eta}) \\ &= 1/(1 + e^{-\theta^T x}) \end{aligned}$$

So, this gives us hypothesis functions of the form $h_{\theta}(x) = 1/(1 + e^{-\theta^T x})$. If you are previously wondering how we came up with the form of the logistic function $1/(1 + e^{-z})$, this gives one answer: Once we assume that y conditioned on x is Bernoulli, it arises as a consequence of the definition of GLMs and exponential family distributions.

To introduce a little more terminology, the function g giving the distribution's mean as a function of the natural parameter ($g(\eta) = E[T(y); \eta]$) is called the **canonical response function**. Its inverse, g^{-1} , is called the **canonical link function**. Thus, the canonical response function for the Gaussian family is just the identity function; and the canonical response function for the Bernoulli is the logistic function.⁷

9.3 Softmax Regression

Let's look at one more example of a GLM. Consider a classification problem in which the response variable y can take on any one of k values, so $y \in \{1, 2, \dots, k\}$. For example, rather than classifying email into the two classes spam or not-spam—which would have been a binary classification problem—we might want to classify it into three classes, such as spam, personal mail, and work-related mail. The response variable is still discrete, but can now take on more than two values. We will thus model it as distributed according to a multinomial distribution.

⁷Many texts use g to denote the link function, and g^{-1} to denote the response function; but the notation we're using here, inherited from the early machine learning literature, will be more consistent with the notation used in the rest of the class.

Let's derive a GLM for modelling this type of multinomial data. To do so, we will begin by expressing the multinomial as an exponential family distribution.

To parameterize a multinomial over k possible outcomes, one could use k parameters $\phi_1, \dots, \phi_k$ specifying the probability of each of the outcomes. However, these parameters would be redundant, or more formally, they would not be independent (since knowing any $k - 1$ of the ϕ_i 's uniquely determines the last one, as they must satisfy $\sum_{i=1}^k \phi_i = 1$). So, we will instead parameterize the multinomial with only $k - 1$ parameters, $\phi_1, \dots, \phi_{k-1}$, where $\phi_i = p(y = i; \phi)$, and $p(y = k; \phi) = 1 - \sum_{i=1}^{k-1} \phi_i$. For notational convenience, we will also let $\phi_k = 1 - \sum_{i=1}^{k-1} \phi_i$, but we should keep in mind that this is not a parameter, and that it is fully specified by $\phi_1, \dots, \phi_{k-1}$.

To express the multinomial as an exponential family distribution, we will define $T(y) \in \mathbb{R}^{k-1}$ as follows:

$$T(1) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, T(2) = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, T(3) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \dots, T(k-1) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}, T(k) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix},$$

Unlike our previous examples, here we do *not* have $T(y) = y$; also, $T(y)$ is now a $k - 1$ dimensional vector, rather than a real number. We will write $(T(y))_i$ to denote the i -th element of the vector $T(y)$.

We introduce one more very useful piece of notation. An indicator function $1\{\cdot\}$ takes on a value of 1 if its argument is true, and 0 otherwise ($1\{\text{True}\} = 1$, $1\{\text{False}\} = 0$). For example, $1\{2 = 3\} = 0$, and $1\{3 = 5 - 2\} = 1$. So, we can also write the relationship between $T(y)$ and y as $(T(y))_i = 1\{y = i\}$. (Before you continue reading, please make sure you understand why this is true!) Further, we have that $E[(T(y))_i] = P(y = i) = \phi_i$.

We are now ready to show that the multinomial is a member of the

exponential family. We have:

$$\begin{aligned}
p(y; \phi) &= \phi_1^{1\{y=1\}} \phi_2^{1\{y=2\}} \dots \phi_k^{1\{y=k\}} \\
&= \phi_1^{1\{y=1\}} \phi_2^{1\{y=2\}} \dots \phi_k^{1 - \sum_{i=1}^{k-1} 1\{y=i\}} \\
&= \phi_1^{(T(y))_1} \phi_2^{(T(y))_2} \dots \phi_k^{1 - \sum_{i=1}^{k-1} (T(y))_i} \\
&= \exp((T(y))_1 \log(\phi_1) + (T(y))_2 \log(\phi_2) + \\
&\quad \dots + \left(1 - \sum_{i=1}^{k-1} (T(y))_i\right) \log(\phi_k)) \\
&= \exp((T(y))_1 \log(\phi_1/\phi_k) + (T(y))_2 \log(\phi_2/\phi_k) + \\
&\quad \dots + (T(y))_{k-1} \log(\phi_{k-1}/\phi_k) + \log(\phi_k)) \\
&= b(y) \exp(\eta^T T(y) - a(\eta))
\end{aligned}$$

where

$$\begin{aligned}
\eta &= \begin{bmatrix} \log(\phi_1/\phi_k) \\ \log(\phi_2/\phi_k) \\ \vdots \\ \log(\phi_{k-1}/\phi_k) \end{bmatrix}, \\
a(\eta) &= -\log(\phi_k) \\
b(y) &= 1.
\end{aligned}$$

This completes our formulation of the multinomial as an exponential family distribution.

The link function is given (for $i = 1, \dots, k$) by

$$\eta_i = \log \frac{\phi_i}{\phi_k}.$$

For convenience, we have also defined $\eta_k = \log(\phi_k/\phi_k) = 0$. To invert the link function and derive the response function, we therefore have that

$$\begin{aligned}
e^{\eta_i} &= \frac{\phi_i}{\phi_k} \\
\phi_k e^{\eta_i} &= \phi_i \\
\phi_k \sum_{i=1}^k e^{\eta_i} &= \sum_{i=1}^k \phi_i = 1
\end{aligned} \tag{7}$$

This implies that $\phi_k = 1/\sum_{i=1}^k e^{\eta_i}$, which can be substituted back into Equation (7) to give the response function

$$\phi_i = \frac{e^{\eta_i}}{\sum_{j=1}^k e^{\eta_j}}$$

This function mapping from the η 's to the ϕ 's is called the **softmax** function.

To complete our model, we use Assumption 3, given earlier, that the η_i 's are linearly related to the x 's. So, have $\eta_i = \theta_i^T x$ (for $i = 1, \dots, k-1$), where $\theta_1, \dots, \theta_{k-1} \in \mathbb{R}^{n+1}$ are the parameters of our model. For notational convenience, we can also define $\theta_k = 0$, so that $\eta_k = \theta_k^T x = 0$, as given previously. Hence, our model assumes that the conditional distribution of y given x is given by

$$\begin{aligned} p(y = i|x; \theta) &= \phi_i \\ &= \frac{e^{\eta_i}}{\sum_{j=1}^k e^{\eta_j}} \\ &= \frac{e^{\theta_i^T x}}{\sum_{j=1}^k e^{\theta_j^T x}} \end{aligned} \tag{8}$$

This model, which applies to classification problems where $y \in \{1, \dots, k\}$, is called **softmax regression**. It is a generalization of logistic regression.

Our hypothesis will output

$$\begin{aligned} h_\theta(x) &= \mathbb{E}[T(y)|x; \theta] \\ &= \mathbb{E} \left[\begin{array}{c|c} \begin{matrix} 1\{y = 1\} \\ 1\{y = 2\} \\ \vdots \\ 1\{y = k-1\} \end{matrix} & x; \theta \end{array} \right] \\ &= \begin{bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_{k-1} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\exp(\theta_1^T x)}{\sum_{j=1}^k \exp(\theta_j^T x)} \\ \frac{\exp(\theta_2^T x)}{\sum_{j=1}^k \exp(\theta_j^T x)} \\ \vdots \\ \frac{\exp(\theta_{k-1}^T x)}{\sum_{j=1}^k \exp(\theta_j^T x)} \end{bmatrix}. \end{aligned}$$

In other words, our hypothesis will output the estimated probability that $p(y = i|x; \theta)$, for every value of $i = 1, \dots, k$. (Even though $h_\theta(x)$ as defined above is only $k-1$ dimensional, clearly $p(y = k|x; \theta)$ can be obtained as $1 - \sum_{i=1}^{k-1} \phi_i$.)

Lastly, let's discuss parameter fitting. Similar to our original derivation of ordinary least squares and logistic regression, if we have a training set of m examples $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$ and would like to learn the parameters θ_i of this model, we would begin by writing down the log-likelihood

$$\begin{aligned}\ell(\theta) &= \sum_{i=1}^m \log p(y^{(i)}|x^{(i)}; \theta) \\ &= \sum_{i=1}^m \log \prod_{l=1}^k \left(\frac{e^{\theta_l^T x^{(i)}}}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \right)^{1_{\{y^{(i)}=l\}}}\end{aligned}$$

To obtain the second line above, we used the definition for $p(y|x; \theta)$ given in Equation (8). We can now obtain the maximum likelihood estimate of the parameters by maximizing $\ell(\theta)$ in terms of θ , using a method such as gradient ascent or Newton's method.

CS229 Lecture notes

Andrew Ng

Part IV

Generative Learning algorithms

So far, we've mainly been talking about learning algorithms that model $p(y|x; \theta)$, the conditional distribution of y given x . For instance, logistic regression modeled $p(y|x; \theta)$ as $h_\theta(x) = g(\theta^T x)$ where g is the sigmoid function. In these notes, we'll talk about a different type of learning algorithm.

Consider a classification problem in which we want to learn to distinguish between elephants ($y = 1$) and dogs ($y = 0$), based on some features of an animal. Given a training set, an algorithm like logistic regression or the perceptron algorithm (basically) tries to find a straight line—that is, a decision boundary—that separates the elephants and dogs. Then, to classify a new animal as either an elephant or a dog, it checks on which side of the decision boundary it falls, and makes its prediction accordingly.

Here's a different approach. First, looking at elephants, we can build a model of what elephants look like. Then, looking at dogs, we can build a separate model of what dogs look like. Finally, to classify a new animal, we can match the new animal against the elephant model, and match it against the dog model, to see whether the new animal looks more like the elephants or more like the dogs we had seen in the training set.

Algorithms that try to learn $p(y|x)$ directly (such as logistic regression), or algorithms that try to learn mappings directly from the space of inputs $\mathcal{X}$ to the labels $\{0, 1\}$, (such as the perceptron algorithm) are called **discriminative** learning algorithms. Here, we'll talk about algorithms that instead try to model $p(x|y)$ (and $p(y)$). These algorithms are called **generative** learning algorithms. For instance, if y indicates whether an example is a dog (0) or an elephant (1), then $p(x|y = 0)$ models the distribution of dogs' features, and $p(x|y = 1)$ models the distribution of elephants' features.

After modeling $p(y)$ (called the **class priors**) and $p(x|y)$, our algorithm

can then use Bayes rule to derive the posterior distribution on y given x :

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}.$$

Here, the denominator is given by $p(x) = p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0)$ (you should be able to verify that this is true from the standard properties of probabilities), and thus can also be expressed in terms of the quantities $p(x|y)$ and $p(y)$ that we've learned. Actually, if we were calculating $p(y|x)$ in order to make a prediction, then we don't actually need to calculate the denominator, since

$$\begin{aligned} \arg \max_y p(y|x) &= \arg \max_y \frac{p(x|y)p(y)}{p(x)} \\ &= \arg \max_y p(x|y)p(y). \end{aligned}$$

1 Gaussian discriminant analysis

The first generative learning algorithm that we'll look at is Gaussian discriminant analysis (GDA). In this model, we'll assume that $p(x|y)$ is distributed according to a multivariate normal distribution. Let's talk briefly about the properties of multivariate normal distributions before moving on to the GDA model itself.

1.1 The multivariate normal distribution

The multivariate normal distribution in n -dimensions, also called the multivariate Gaussian distribution, is parameterized by a **mean vector** $\mu \in \mathbb{R}^n$ and a **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$, where $\Sigma \geq 0$ is symmetric and positive semi-definite. Also written " $\mathcal{N}(\mu, \Sigma)$ ", its density is given by:

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right).$$

In the equation above, " $|\Sigma|$ " denotes the determinant of the matrix Σ .

For a random variable X distributed $\mathcal{N}(\mu, \Sigma)$, the mean is (unsurprisingly) given by μ :

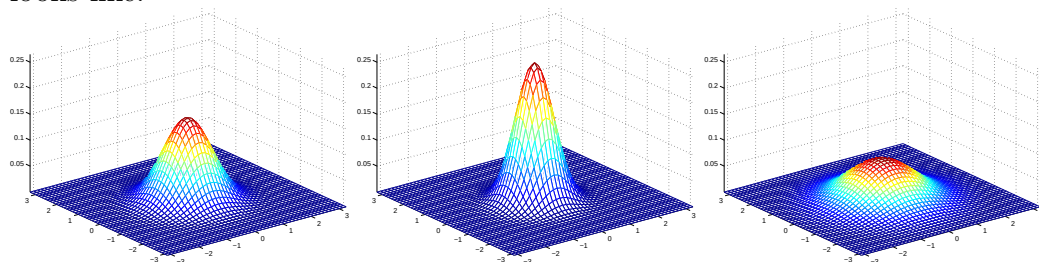
$$\mathbb{E}[X] = \int_x x p(x; \mu, \Sigma) dx = \mu$$

The **covariance** of a vector-valued random variable Z is defined as $\text{Cov}(Z) = \mathbb{E}[(Z - \mathbb{E}[Z])(Z - \mathbb{E}[Z])^T]$. This generalizes the notion of the variance of a

real-valued random variable. The covariance can also be defined as $\text{Cov}(Z) = E[ZZ^T] - (E[Z])(E[Z])^T$. (You should be able to prove to yourself that these two definitions are equivalent.) If $X \sim \mathcal{N}(\mu, \Sigma)$, then

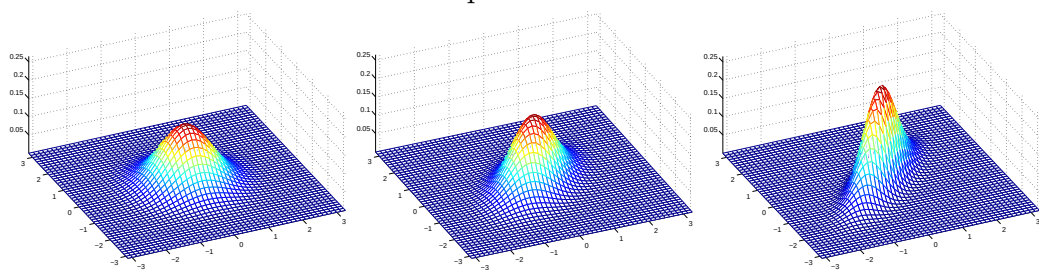
$$\text{Cov}(X) = \Sigma.$$

Here're some examples of what the density of a Gaussian distribution looks like:



The left-most figure shows a Gaussian with mean zero (that is, the 2x1 zero-vector) and covariance matrix $\Sigma = I$ (the 2x2 identity matrix). A Gaussian with zero mean and identity covariance is also called the **standard normal distribution**. The middle figure shows the density of a Gaussian with zero mean and $\Sigma = 0.6I$; and in the rightmost figure shows one with $\Sigma = 2I$. We see that as Σ becomes larger, the Gaussian becomes more “spread-out,” and as it becomes smaller, the distribution becomes more “compressed.”

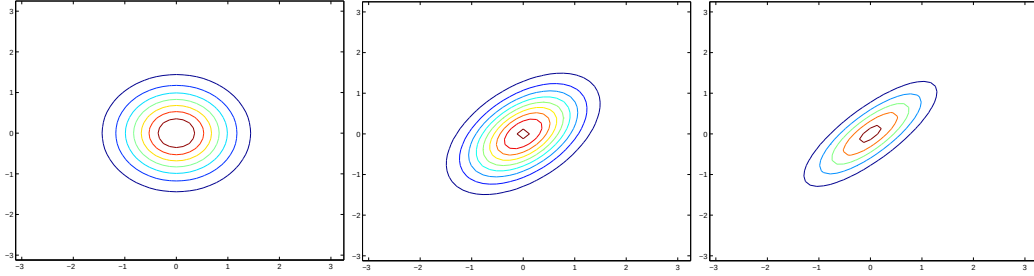
Let's look at some more examples.



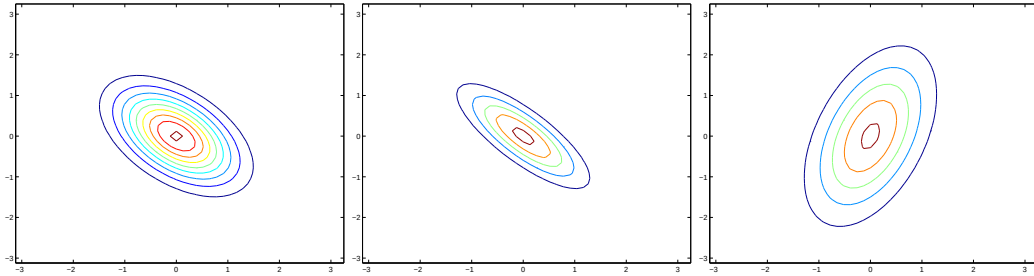
The figures above show Gaussians with mean 0, and with covariance matrices respectively

$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

The leftmost figure shows the familiar standard normal distribution, and we see that as we increase the off-diagonal entry in Σ , the density becomes more “compressed” towards the 45° line (given by $x_1 = x_2$). We can see this more clearly when we look at the contours of the same three densities:



Here's one last set of examples generated by varying Σ :

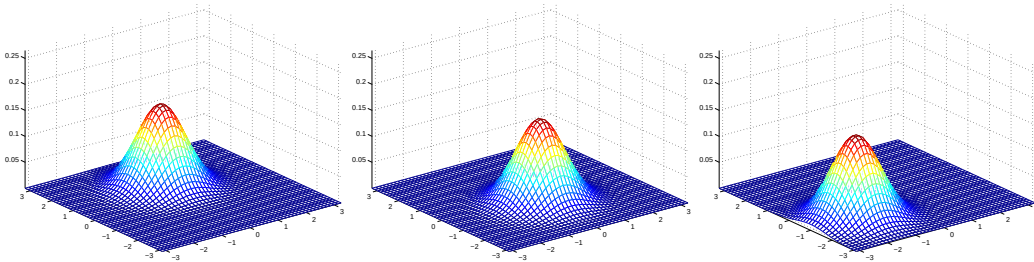


The plots above used, respectively,

$$\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}.$$

From the leftmost and middle figures, we see that by decreasing the off-diagonal elements of the covariance matrix, the density now becomes “compressed” again, but in the opposite direction. Lastly, as we vary the parameters, more generally the contours will form ellipses (the rightmost figure showing an example).

As our last set of examples, fixing $\Sigma = I$, by varying μ , we can also move the mean of the density around.



The figures above were generated using $\Sigma = I$, and respectively

$$\mu = \begin{bmatrix} 1 \\ 0 \end{bmatrix}; \quad \mu = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}; \quad \mu = \begin{bmatrix} -1 \\ -1.5 \end{bmatrix}.$$

1.2 The Gaussian Discriminant Analysis model

When we have a classification problem in which the input features x are continuous-valued random variables, we can then use the Gaussian Discriminant Analysis (GDA) model, which models $p(x|y)$ using a multivariate normal distribution. The model is:

$$\begin{aligned} y &\sim \text{Bernoulli}(\phi) \\ x|y=0 &\sim \mathcal{N}(\mu_0, \Sigma) \\ x|y=1 &\sim \mathcal{N}(\mu_1, \Sigma) \end{aligned}$$

Writing out the distributions, this is:

$$\begin{aligned} p(y) &= \phi^y(1-\phi)^{1-y} \\ p(x|y=0) &= \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_0)^T\Sigma^{-1}(x-\mu_0)\right) \\ p(x|y=1) &= \frac{1}{(2\pi)^{n/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_1)^T\Sigma^{-1}(x-\mu_1)\right) \end{aligned}$$

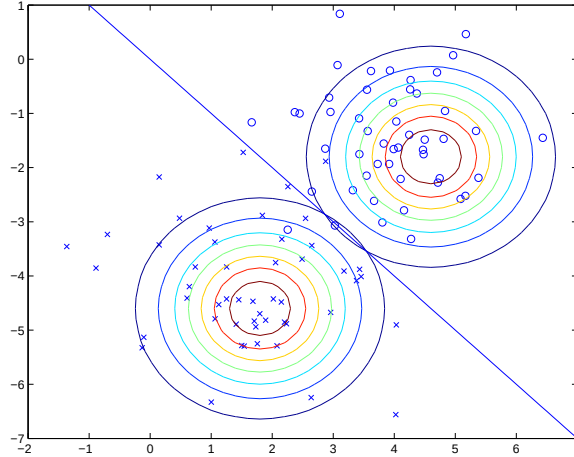
Here, the parameters of our model are ϕ , Σ , μ_0 and μ_1 . (Note that while there're two different mean vectors μ_0 and μ_1 , this model is usually applied using only one covariance matrix Σ .) The log-likelihood of the data is given by

$$\begin{aligned} \ell(\phi, \mu_0, \mu_1, \Sigma) &= \log \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \\ &= \log \prod_{i=1}^m p(x^{(i)}|y^{(i)}; \mu_0, \mu_1, \Sigma) p(y^{(i)}; \phi). \end{aligned}$$

By maximizing ℓ with respect to the parameters, we find the maximum likelihood estimate of the parameters (see problem set 1) to be:

$$\begin{aligned}\phi &= \frac{1}{m} \sum_{i=1}^m 1\{y^{(i)} = 1\} \\ \mu_0 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \\ \mu_1 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\} x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \\ \Sigma &= \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T.\end{aligned}$$

Pictorially, what the algorithm is doing can be seen in as follows:



Shown in the figure are the training set, as well as the contours of the two Gaussian distributions that have been fit to the data in each of the two classes. Note that the two Gaussians have contours that are the same shape and orientation, since they share a covariance matrix Σ , but they have different means μ_0 and μ_1 . Also shown in the figure is the straight line giving the decision boundary at which $p(y = 1|x) = 0.5$. On one side of the boundary, we'll predict $y = 1$ to be the most likely outcome, and on the other side, we'll predict $y = 0$.

1.3 Discussion: GDA and logistic regression

The GDA model has an interesting relationship to logistic regression. If we view the quantity $p(y = 1|x; \phi, \mu_0, \mu_1, \Sigma)$ as a function of x , we'll find that it

can be expressed in the form

$$p(y = 1|x; \phi, \Sigma, \mu_0, \mu_1) = \frac{1}{1 + \exp(-\theta^T x)},$$

where θ is some appropriate function of $\phi, \Sigma, \mu_0, \mu_1$.¹ This is exactly the form that logistic regression—a discriminative algorithm—used to model $p(y = 1|x)$.

When would we prefer one model over another? GDA and logistic regression will, in general, give different decision boundaries when trained on the same dataset. Which is better?

We just argued that if $p(x|y)$ is multivariate gaussian (with shared Σ), then $p(y|x)$ necessarily follows a logistic function. The converse, however, is not true; i.e., $p(y|x)$ being a logistic function does not imply $p(x|y)$ is multivariate gaussian. This shows that GDA makes *stronger* modeling assumptions about the data than does logistic regression. It turns out that when these modeling assumptions are correct, then GDA will find better fits to the data, and is a better model. Specifically, when $p(x|y)$ is indeed gaussian (with shared Σ), then GDA is **asymptotically efficient**. Informally, this means that in the limit of very large training sets (large m), there is no algorithm that is strictly better than GDA (in terms of, say, how accurately they estimate $p(y|x)$). In particular, it can be shown that in this setting, GDA will be a better algorithm than logistic regression; and more generally, even for small training set sizes, we would generally expect GDA to better.

In contrast, by making significantly weaker assumptions, logistic regression is also more *robust* and less sensitive to incorrect modeling assumptions. There are many different sets of assumptions that would lead to $p(y|x)$ taking the form of a logistic function. For example, if $x|y = 0 \sim \text{Poisson}(\lambda_0)$, and $x|y = 1 \sim \text{Poisson}(\lambda_1)$, then $p(y|x)$ will be logistic. Logistic regression will also work well on Poisson data like this. But if we were to use GDA on such data—and fit Gaussian distributions to such non-Gaussian data—then the results will be less predictable, and GDA may (or may not) do well.

To summarize: GDA makes stronger modeling assumptions, and is more data efficient (i.e., requires less training data to learn “well”) when the modeling assumptions are correct or at least approximately correct. Logistic regression makes weaker assumptions, and is significantly more robust to deviations from modeling assumptions. Specifically, when the data is indeed non-Gaussian, then in the limit of large datasets, logistic regression will

¹This uses the convention of redefining the $x^{(i)}$'s on the right-hand-side to be $n + 1$ -dimensional vectors by adding the extra coordinate $x_0^{(i)} = 1$; see problem set 1.

almost always do better than GDA. For this reason, in practice logistic regression is used more often than GDA. (Some related considerations about discriminative vs. generative models also apply for the Naive Bayes algorithm that we discuss next, but the Naive Bayes algorithm is still considered a very good, and is certainly also a very popular, classification algorithm.)

2 Naive Bayes

In GDA, the feature vectors x were continuous, real-valued vectors. Let's now talk about a different learning algorithm in which the x_i 's are discrete-valued.

For our motivating example, consider building an email spam filter using machine learning. Here, we wish to classify messages according to whether they are unsolicited commercial (spam) email, or non-spam email. After learning to do this, we can then have our mail reader automatically filter out the spam messages and perhaps place them in a separate mail folder. Classifying emails is one example of a broader set of problems called **text classification**.

Let's say we have a training set (a set of emails labeled as spam or non-spam). We'll begin our construction of our spam filter by specifying the features x_i used to represent an email.

We will represent an email via a feature vector whose length is equal to the number of words in the dictionary. Specifically, if an email contains the i -th word of the dictionary, then we will set $x_i = 1$; otherwise, we let $x_i = 0$. For instance, the vector

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{array}{l} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{array}$$

is used to represent an email that contains the words “a” and “buy,” but not “aardvark,” “aardwolf” or “zygmurgy.”² The set of words encoded into the

²Actually, rather than looking through an english dictionary for the list of all english words, in practice it is more common to look through our training set and encode in our feature vector only the words that occur at least once there. Apart from reducing the

feature vector is called the **vocabulary**, so the dimension of x is equal to the size of the vocabulary.

Having chosen our feature vector, we now want to build a generative model. So, we have to model $p(x|y)$. But if we have, say, a vocabulary of 50000 words, then $x \in \{0, 1\}^{50000}$ (x is a 50000-dimensional vector of 0's and 1's), and if we were to model x explicitly with a multinomial distribution over the 2^{50000} possible outcomes, then we'd end up with a $(2^{50000} - 1)$ -dimensional parameter vector. This is clearly too many parameters.

To model $p(x|y)$, we will therefore make a very strong assumption. We will assume that the x_i 's are conditionally independent given y . This assumption is called the **Naive Bayes (NB) assumption**, and the resulting algorithm is called the **Naive Bayes classifier**. For instance, if $y = 1$ means spam email; "buy" is word 2087 and "price" is word 39831; then we are assuming that if I tell you $y = 1$ (that a particular piece of email is spam), then knowledge of x_{2087} (knowledge of whether "buy" appears in the message) will have no effect on your beliefs about the value of x_{39831} (whether "price" appears). More formally, this can be written $p(x_{2087}|y) = p(x_{2087}|y, x_{39831})$. (Note that this is *not* the same as saying that x_{2087} and x_{39831} are independent, which would have been written " $p(x_{2087}) = p(x_{2087}|x_{39831})$ "; rather, we are only assuming that x_{2087} and x_{39831} are conditionally independent *given* y .)

We now have:

$$\begin{aligned} p(x_1, \dots, x_{50000}|y) &= p(x_1|y)p(x_2|y, x_1)p(x_3|y, x_1, x_2) \cdots p(x_{50000}|y, x_1, \dots, x_{49999}) \\ &= p(x_1|y)p(x_2|y)p(x_3|y) \cdots p(x_{50000}|y) \\ &= \prod_{i=1}^n p(x_i|y) \end{aligned}$$

The first equality simply follows from the usual properties of probabilities, and the second equality used the NB assumption. We note that even though the Naive Bayes assumption is an extremely strong assumptions, the resulting algorithm works well on many problems.

number of words modeled and hence reducing our computational and space requirements, this also has the advantage of allowing us to model/include as a feature many words that may appear in your email (such as "cs229") but that you won't find in a dictionary. Sometimes (as in the homework), we also exclude the very high frequency words (which will be words like "the," "of," "and,"; these high frequency, "content free" words are called **stop words**) since they occur in so many documents and do little to indicate whether an email is spam or non-spam.

Our model is parameterized by $\phi_{i|y=1} = p(x_i = 1|y = 1)$, $\phi_{i|y=0} = p(x_i = 1|y = 0)$, and $\phi_y = p(y = 1)$. As usual, given a training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$, we can write down the joint likelihood of the data:

$$\mathcal{L}(\phi_y, \phi_{j|y=0}, \phi_{j|y=1}) = \prod_{i=1}^m p(x^{(i)}, y^{(i)}).$$

Maximizing this with respect to $\phi_y, \phi_{i|y=0}$ and $\phi_{i|y=1}$ gives the maximum likelihood estimates:

$$\begin{aligned}\phi_{j|y=1} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \\ \phi_{j|y=0} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \\ \phi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m}\end{aligned}$$

In the equations above, the “ $\wedge$ ” symbol means “and.” The parameters have a very natural interpretation. For instance, $\phi_{j|y=1}$ is just the fraction of the spam ($y = 1$) emails in which word j does appear.

Having fit all these parameters, to make a prediction on a new example with features x , we then simply calculate

$$\begin{aligned}p(y = 1|x) &= \frac{p(x|y = 1)p(y = 1)}{p(x)} \\ &= \frac{(\prod_{i=1}^n p(x_i|y = 1)) p(y = 1)}{(\prod_{i=1}^n p(x_i|y = 1)) p(y = 1) + (\prod_{i=1}^n p(x_i|y = 0)) p(y = 0)},\end{aligned}$$

and pick whichever class has the higher posterior probability.

Lastly, we note that while we have developed the Naive Bayes algorithm mainly for the case of problems where the features x_i are binary-valued, the generalization to where x_i can take values in $\{1, 2, \dots, k_i\}$ is straightforward. Here, we would simply model $p(x_i|y)$ as multinomial rather than as Bernoulli. Indeed, even if some original input attribute (say, the living area of a house, as in our earlier example) were continuous valued, it is quite common to **discretize** it—that is, turn it into a small set of discrete values—and apply Naive Bayes. For instance, if we use some feature x_i to represent living area, we might discretize the continuous values as follows:

Living area (sq. feet)	< 400	400-800	800-1200	1200-1600	>1600
x_i	1	2	3	4	5

Thus, for a house with living area 890 square feet, we would set the value of the corresponding feature x_i to 3. We can then apply the Naive Bayes algorithm, and model $p(x_i|y)$ with a multinomial distribution, as described previously. When the original, continuous-valued attributes are not well-modeled by a multivariate normal distribution, discretizing the features and using Naive Bayes (instead of GDA) will often result in a better classifier.

2.1 Laplace smoothing

The Naive Bayes algorithm as we have described it will work fairly well for many problems, but there is a simple change that makes it work much better, especially for text classification. Let's briefly discuss a problem with the algorithm in its current form, and then talk about how we can fix it.

Consider spam/email classification, and let's suppose that, after completing CS229 and having done excellent work on the project, you decide around June 2003 to submit the work you did to the NIPS conference for publication. (NIPS is one of the top machine learning conferences, and the deadline for submitting a paper is typically in late June or early July.) Because you end up discussing the conference in your emails, you also start getting messages with the word "nips" in it. But this is your first NIPS paper, and until this time, you had not previously seen any emails containing the word "nips"; in particular "nips" did not ever appear in your training set of spam/non-spam emails. Assuming that "nips" was the 35000th word in the dictionary, your Naive Bayes spam filter therefore had picked its maximum likelihood estimates of the parameters $\phi_{35000|y}$ to be

$$\begin{aligned}\phi_{35000|y=1} &= \frac{\sum_{i=1}^m 1\{x_{35000}^{(i)} = 1 \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} = 0 \\ \phi_{35000|y=0} &= \frac{\sum_{i=1}^m 1\{x_{35000}^{(i)} = 1 \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} = 0\end{aligned}$$

I.e., because it has never seen "nips" before in either spam or non-spam training examples, it thinks the probability of seeing it in either type of email is zero. Hence, when trying to decide if one of these messages containing "nips" is spam, it calculates the class posterior probabilities, and obtains

$$\begin{aligned}p(y = 1|x) &= \frac{\prod_{i=1}^n p(x_i|y = 1)p(y = 1)}{\prod_{i=1}^n p(x_i|y = 1)p(y = 1) + \prod_{i=1}^n p(x_i|y = 0)p(y = 0)} \\ &= \frac{0}{0}.\end{aligned}$$

This is because each of the terms “ $\prod_{i=1}^n p(x_i|y)$ ” includes a term $p(x_{35000}|y) = 0$ that is multiplied into it. Hence, our algorithm obtains 0/0, and doesn’t know how to make a prediction.

Stating the problem more broadly, it is statistically a bad idea to estimate the probability of some event to be zero just because you haven’t seen it before in your finite training set. Take the problem of estimating the mean of a multinomial random variable z taking values in $\{1, \dots, k\}$. We can parameterize our multinomial with $\phi_i = p(z = i)$. Given a set of m independent observations $\{z^{(1)}, \dots, z^{(m)}\}$, the maximum likelihood estimates are given by

$$\phi_j = \frac{\sum_{i=1}^m 1\{z^{(i)} = j\}}{m}.$$

As we saw previously, if we were to use these maximum likelihood estimates, then some of the ϕ_j ’s might end up as zero, which was a problem. To avoid this, we can use **Laplace smoothing**, which replaces the above estimate with

$$\phi_j = \frac{\sum_{i=1}^m 1\{z^{(i)} = j\} + 1}{m + k}.$$

Here, we’ve added 1 to the numerator, and k to the denominator. Note that $\sum_{j=1}^k \phi_j = 1$ still holds (check this yourself!), which is a desirable property since the ϕ_j ’s are estimates for probabilities that we know must sum to 1. Also, $\phi_j \neq 0$ for all values of j , solving our problem of probabilities being estimated as zero. Under certain (arguably quite strong) conditions, it can be shown that the Laplace smoothing actually gives the optimal estimator of the ϕ_j ’s.

Returning to our Naive Bayes classifier, with Laplace smoothing, we therefore obtain the following estimates of the parameters:

$$\begin{aligned}\phi_{j|y=1} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 1\} + 1}{\sum_{i=1}^m 1\{y^{(i)} = 1\} + 2} \\ \phi_{j|y=0} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1 \wedge y^{(i)} = 0\} + 1}{\sum_{i=1}^m 1\{y^{(i)} = 0\} + 2}\end{aligned}$$

(In practice, it usually doesn’t matter much whether we apply Laplace smoothing to ϕ_y or not, since we will typically have a fair fraction each of spam and non-spam messages, so ϕ_y will be a reasonable estimate of $p(y = 1)$ and will be quite far from 0 anyway.)

2.2 Event models for text classification

To close off our discussion of generative learning algorithms, let's talk about one more model that is specifically for text classification. While Naive Bayes as we've presented it will work well for many classification problems, for text classification, there is a related model that does even better.

In the specific context of text classification, Naive Bayes as presented uses the what's called the **multi-variate Bernoulli event model**. In this model, we assumed that the way an email is generated is that first it is randomly determined (according to the class priors $p(y)$) whether a spammer or non-spammer will send you your next message. Then, the person sending the email runs through the dictionary, deciding whether to include each word i in that email independently and according to the probabilities $p(x_i = 1|y) = \phi_{i|y}$. Thus, the probability of a message was given by $p(y) \prod_{i=1}^n p(x_i|y)$.

Here's a different model, called the **multinomial event model**. To describe this model, we will use a different notation and set of features for representing emails. We let x_i denote the identity of the i -th word in the email. Thus, x_i is now an integer taking values in $\{1, \dots, |V|\}$, where $|V|$ is the size of our vocabulary (dictionary). An email of n words is now represented by a vector $(x_1, x_2, \dots, x_n)$ of length n ; note that n can vary for different documents. For instance, if an email starts with "A NIPS . . .," then $x_1 = 1$ ("a" is the first word in the dictionary), and $x_2 = 35000$ (if "nips" is the 35000th word in the dictionary).

In the multinomial event model, we assume that the way an email is generated is via a random process in which spam/non-spam is first determined (according to $p(y)$) as before. Then, the sender of the email writes the email by first generating x_1 from some multinomial distribution over words ($p(x_1|y)$). Next, the second word x_2 is chosen independently of x_1 but from the same multinomial distribution, and similarly for x_3, x_4 , and so on, until all n words of the email have been generated. Thus, the overall probability of a message is given by $p(y) \prod_{i=1}^n p(x_i|y)$. Note that this formula looks like the one we had earlier for the probability of a message under the multi-variate Bernoulli event model, but that the terms in the formula now mean very different things. In particular $x_i|y$ is now a multinomial, rather than a Bernoulli distribution.

The parameters for our new model are $\phi_y = p(y)$ as before, $\phi_{k|y=1} = p(x_j = k|y = 1)$ (for any j) and $\phi_{i|y=0} = p(x_j = k|y = 0)$. Note that we have assumed that $p(x_j|y)$ is the same for all values of j (i.e., that the distribution according to which a word is generated does not depend on its position j within the email).

If we are given a training set $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$ where $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_{n_i}^{(i)})$ (here, n_i is the number of words in the i -training example), the likelihood of the data is given by

$$\begin{aligned}\mathcal{L}(\phi, \phi_{k|y=0}, \phi_{k|y=1}) &= \prod_{i=1}^m p(x^{(i)}, y^{(i)}) \\ &= \prod_{i=1}^m \left(\prod_{j=1}^{n_i} p(x_j^{(i)} | y; \phi_{k|y=0}, \phi_{k|y=1}) \right) p(y^{(i)}; \phi_y).\end{aligned}$$

Maximizing this yields the maximum likelihood estimates of the parameters:

$$\begin{aligned}\phi_{k|y=1} &= \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\} n_i} \\ \phi_{k|y=0} &= \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 0\}}{\sum_{i=1}^m 1\{y^{(i)} = 0\} n_i} \\ \phi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m}.\end{aligned}$$

If we were to apply Laplace smoothing (which needed in practice for good performance) when estimating $\phi_{k|y=0}$ and $\phi_{k|y=1}$, we add 1 to the numerators and $|V|$ to the denominators, and obtain:

$$\begin{aligned}\phi_{k|y=1} &= \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 1\} + 1}{\sum_{i=1}^m 1\{y^{(i)} = 1\} n_i + |V|} \\ \phi_{k|y=0} &= \frac{\sum_{i=1}^m \sum_{j=1}^{n_i} 1\{x_j^{(i)} = k \wedge y^{(i)} = 0\} + 1}{\sum_{i=1}^m 1\{y^{(i)} = 0\} n_i + |V|}.\end{aligned}$$

While not necessarily the very best classification algorithm, the Naive Bayes classifier often works surprisingly well. It is often also a very good “first thing to try,” given its simplicity and ease of implementation.